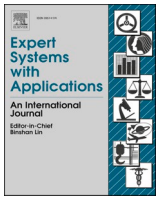
Expert Systems With Applications 332 (2027) 133653

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa



AL-WOA: adaptive learning whale optimization algorithm for energy-aware load balancing in cloud data centers

Jyoti Ranjan Nayak, Subasish Mohapatra, Ashutosh Mohapatra *

School of Computer Sciences, Odisha University of Technology & Research, Bhubaneswar, Odisha, India

ARTICLE INFO

Keywords:
Adaptive learning
Cloud resource management
Energy efficiency
Reinforcement learning
Whale Optimization Algorithm (WOA)

ABSTRACT

Balancing energy efficiency with service quality in cloud data centers presents a fundamental optimization challenge where traditional resource management approaches fail to adapt to dynamic workload conditions. This paper proposes AL-WOA, a hybrid optimization framework combining the Whale Optimization Algorithm (WOA) with adaptive reinforcement learning (RL) for energy-aware and performance-conscious load balancing and virtual machine consolidation. The approach employs reinforcement learning to continuously adjust metaheuristic parameters, enabling dynamic regulation of exploration–exploitation trade-offs based on real-time system states. Comprehensive evaluation using real-world workload traces from Google and Alibaba clusters, Hadoop benchmarks, and synthetic scenarios demonstrates that AL-WOA consistently outperforms classical heuristics, standalone metaheuristics, and pure reinforcement learning methods across energy consumption, SLA compliance, and response time metrics. Ablation analysis confirms the hybrid architecture delivers faster convergence and superior solution quality compared to individual components operating independently, with measurable synergistic benefits beyond simple combination. The findings establish AL-WOA's viability for production cloud deployments, offering a practical framework for sustainable infrastructure management addressing both environmental objectives and operational performance requirements in modern data centers.

1. Introduction

Cloud data centers consume approximately 1–2% of global electricity, a share projected to grow substantially as cloud adoption accelerates (Beloglazov & Buyya, 2013; Dayarathna, Wen, & Fan, 2016). Balancing computational performance with energy efficiency is critical for reducing operational costs and carbon emissions (Nathuji & Schwan, 2007; Singh & Buyya, 2024). A central challenge is optimal virtual machine (VM) placement under dynamic workloads while satisfying Service Level Agreements (SLAs) – any misconfiguration leads to either energy waste through under-utilization or SLA violations through resource contention (Gupta & Vahid, 2018; Li & Wu, 2017). Metaheuristics such as WOA, PSO, and GA offer effective global search for VM placement and task scheduling (Mirjalili & Lewis, 2016; Qureshi, 2023), yet rely on static, pre-configured parameters that cannot respond to workload fluctuations, causing premature convergence or oscillation. Reinforcement learning (RL) addresses this through continuous policy learning from environmental feedback (Al-Tarawneh, Mahmud, & Shojafar, 2024; Mao & Li, 2019), but standalone RL suffers from slow convergence and training instability at cloud scale (Li, Wang, & Chen, 2024).

Recent hybrid attempts have combined metaheuristics with learning methods – notably PSO-DRL (Das, Dey, & Ghosh, 2025), GA-PPO (Wang, 2024), and QL-WOA (Elaziz, 2024) – but these exhibit three persistent limitations. First, coupling is loose: the learning component supplies occasional parameter suggestions rather than continuously adapting metaheuristic behavior during execution. Second, parameter adaptation is static between decision intervals, leaving the search strategy unresponsive to intra-interval workload shifts. Third, none simultaneously optimize energy consumption, SLA compliance, and migration cost within a single tightly integrated feedback loop – they address these objectives separately or with fixed weight schedules. These gaps motivate a fundamentally different integration paradigm.

This paper proposes AL-WOA, a hybrid framework that embeds a PPO-based RL agent directly into WOA's iterative search loop. The agent continuously observes real-time system state and adjusts WOA's core parameters – convergence coefficient a, exploitation vector $\vec{A}$, and exploration vector $\vec{C}$ – at every iteration rather than at fixed intervals.

* Corresponding author.
E-mail address: ashutosh.mohapatra055@gmail.com (A. Mohapatra).

https://doi.org/10.1016/j.eswa.2026.133653
Received 16 February 2026; Received in revised form 24 May 2026; Accepted 12 July 2026
Available online 14 July 2026

This tight coupling enables dynamic regulation of exploration–exploitation trade-offs in direct response to workload conditions. The primary contributions are:

1. *Tightly coupled WOA-PPO architecture:* a formally defined closed-loop integration where RL actions map continuously to WOA search parameters with explicit boundary constraints, distinguishing it from loose-coupling hybrids (Das et al., 2025; Elaziz, 2024; Wang, 2024).
2. *Multi-objective adaptive optimization:* a unified reward signal simultaneously penalizing energy consumption, SLA violations, and migration cost, enabling the agent to discover Pareto-optimal consolidation policies without manual retuning across workload regimes.
3. *Comprehensive empirical validation:* evaluation on four real-world and benchmark workload traces – Google Cluster, Alibaba Production, HiBench, and synthetic bursty patterns – with ablation analysis quantifying synergistic gains beyond individual component contributions.

2. Related work

2.1. WOA and its variants for cloud resource management

The Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016), inspired by humpback whale bubble-net hunting, has gained traction in cloud computing for VM placement and task scheduling due to its effective exploration–exploitation balance (Ahuja & Singh, 2022; Kaur & Arora, 2020). Extensions addressing energy and makespan include chaotic WOA (Kaur & Arora, 2020), which improves diversity through chaotic maps, multi-objective WOA (Zhang & Xu, 2020) for simultaneous energy-makespan optimization, and self-adaptive WOA (Abdel-Basset & Mohamed, 2020) incorporating mechanism-switching strategies. However, these variants predate 2021 and share a fundamental limitation: parameter schedules are fixed prior to execution and cannot respond to runtime workload changes.

Recent work has introduced more sophisticated adaptations. Opposition-based WOA (Ghoneim, 2023) accelerates convergence by evaluating candidate solutions and their opposites simultaneously, improving solution diversity in early iterations. Lévy-flight enhanced WOA (Zhao, 2024) replaces standard random walks with heavy-tailed Lévy distributions, enabling longer escape jumps from local optima during bursty workload phases. Multi-strategy adaptive WOA (Abdel-Basset & Shawky, 2022) switches between hunting behaviors based on fitness stagnation detection, while self-learning WOA (Alharbi, 2025) incorporates internal reward signals to autonomously adjust exploration rates. These variants demonstrate measurable improvements – 10–15% energy reduction and 8–12% latency improvement over standard WOA (Alharbi, 2025) – yet share a critical architectural constraint: adaptation is driven by internal heuristic rules or fixed fitness thresholds rather than observed system state. None incorporate feedback from SLA violation rates, queue lengths, or energy trends to guide parameter adjustment, leaving them unable to distinguish between workload regimes that superficially resemble each other in fitness space but demand different search strategies.

2.2. Reinforcement learning for cloud scheduling

RL-based schedulers have been applied to job scheduling (Mao & Li, 2019), dynamic VM consolidation (Xu & Wang, 2021), and SLA-aware resource management (Gupta & Vahid, 2018). DQN (Mnih, 2015) and PPO (Schulman, 1707) demonstrate ability to learn complex scheduling policies outperforming static heuristics in non-stationary environments. Recent work shows DQN-based schedulers reduce SLA violations and energy consumption simultaneously (Xu, Huang, & Zhang, 2023), while PPO-based frameworks achieve up to 18% energy improvement through adaptive VM migration (Zhang, Lin, & Liu, 2025). Multi-agent RL (MARL) extends these gains to large-scale multi-datacenter settings (Qian, 2024).

Standalone RL, however, faces inherent limitations in cloud scheduling contexts. Convergence is slow in high-dimensional state spaces, exploration costs are prohibitive during training, and learned policies overfit to training workload distributions, degrading under unseen patterns (Li et al., 2024). These limitations motivate hybridization with metaheuristics that provide structured global search to complement RL's policy learning.

2.3. Hybrid RL–metaheuristic frameworks

Hybrid RL-metaheuristic approaches have emerged across four principal families, each attempting to combine global search with adaptive learning (Simaiya, Nayyar, & Buyya, 2024).

RL-PSO hybrids use RL to tune PSO's inertia weight and acceleration coefficients dynamically. DDPG-PSO (Das et al., 2025) and DRL-PSO (Ghafir, 2024) achieve faster convergence than standalone PSO by steering particle updates based on reward signals. However, RL operates at the population level, adjusting hyperparameters between epochs rather than within each iteration, constituting loose coupling.

RL-GA hybrids apply RL to adapt crossover and mutation rates. DQN-GA (El-Kenawy, 2023) achieves 12–15% energy reduction over standalone DQN in simulations, and PPO-GA (Wang, 2024) uses genetic diversity to improve policy exploration. These approaches suffer from high computational cost as population size scales, and parameter adaptation occurs between generations rather than continuously.

RL-ACO hybrids use RL to adaptively control pheromone evaporation and deposit rates (Xu, 2023), achieving 18% makespan reduction and 15% energy savings over standard ACO (Sharma, 2023). Adaptation is event-driven rather than state-continuous – pheromone parameters update only when convergence stagnation is detected.

RL-WOA hybrids represent the closest family to AL-WOA. QL-WOA (Elaziz, 2024) uses Q-learning to update WOA search coefficients, reducing energy consumption by up to 17% over standard WOA. DRL-WOA (Kaur & Singh, 2024) extends this with deep RL to control whale population dynamics, maintaining SLA compliance under extreme workload variation. Self-learning WOA (Alharbi, 2025) incorporates RL-inspired internal rewards for autonomous parameter adjustment. Despite these advances, existing RL-WOA approaches share two unresolved limitations: parameter updates occur at fixed decision intervals (not every iteration), and the RL agent's action space covers only a subset of WOA's governing parameters – typically either a or $\vec{A}$, but not both simultaneously with $\vec{C}$. This partial coupling leaves residual static behavior in the search mechanism.

2.4. Baseline scheduling heuristics

Classical baselines including Round Robin (Chhabra, 2024), Least Load (Li & Zhu, 2017), and Energy-Aware Greedy (EAG) (Beloglazov, Abawajy, & Buyya, 2012) are computationally efficient but apply fixed allocation logic without workload awareness, causing systematic energy waste and SLA instability under heterogeneous loads. Advanced metaheuristics PSO (Qureshi, 2023) and GA (Gupta, Kumari, & Singh, 2024) improve solution quality but remain susceptible to local optima without adaptive mechanisms. These baselines serve as lower-bound comparators in experimental evaluation.

2.5. Research gap analysis

Table 1 maps identified limitations across existing approaches to the specific AL-WOA components that address them.

The gaps in Table 1 collectively motivate AL-WOA's design: a tightly coupled hybrid where PPO-based RL continuously guides all three WOA search parameters using a multi-objective reward signal derived from

Table 1
Research gap mapping to AL-WOA components.

Gap	Affected Approaches	AL-WOA Component Addressing It
Static parameter schedules; no runtime adaptation	Standard WOA, PSO, GA, ACO variants	PPO agent continuously adjusts $a, \vec{A}, \vec{C}$ every iteration based on observed state
Loose coupling; RL updates parameters between epochs only	RL-PSO, RL-GA, RL-ACO hybrids	Closed-loop integration: RL action applied within each WOA position update cycle
Partial parameter coverage; only subset of WOA parameters adapted	QL-WOA, DRL-WOA, SL-WOA	Action space $A_t = a, \vec{A}, \vec{C}$ covers all three governing parameters simultaneously
Single-objective or sequentially weighted optimization	EAG, most WOA variants	Unified reward $R_t = -(w_1 E_{total} + w_2 SLAV + w_3 C_{mig})$ jointly penalizes all three objectives
Slow RL convergence; instability in large-scale settings	Standalone RL (DQN, PPO, DDPG)	WOA population provides structured candidate solutions, reducing RL exploration burden
No validation across diverse real-world workload traces	Most hybrid approaches test on synthetic traces only	Evaluated on Google Cluster, Alibaba Production, HiBench, and synthetic bursty traces

real-time system state.

3. Methodology

3.1. Mathematical formulation of energy-aware load balancing with SLA and migration constraints

The energy-aware load balancing problem in cloud data centers is modeled as a constrained multi-objective optimization involving VM placement and migration. A cloud data center comprises physical hosts $H = h_1, h_2, \cdots, h_m$ with specific CPU, memory, and bandwidth capacities. The workload consists of VMs $V = v_1, v_2, \cdots, v_n$, each demanding computational resources. Host resource utilization at time t is defined as:

$$U_j(t) = \frac{\sum_{v_i \in V_j} CPU(v_i)}{CPU(h_j)} \tag{1}$$

where V_j denotes VMs assigned to host h_j. Excessive utilization causes SLA violations from resource contention, while under-utilization wastes energy (Beloglazov & Buyya, 2012; Xu, Huang, & Chen, 2020).

Power consumption follows a linear relationship with utilization, validated through empirical studies (Barroso, Hölzle, & Ranganathan, 2018):

$$P_j(t) = P_j^{idle} + \left(P_j^{max} - P_j^{idle}\right) \cdot U_j(t) \tag{2}$$

where P_j^{idle} and P_j^{max} represent idle and maximum power consumption. This model captures that servers consume 50–70% of peak power even when idle (Dayarathna et al., 2016).

SLA violations occur when resources are insufficient or excessive migrations degrade performance. The violation rate is:

$$SLAV = \frac{Numberofviolatedrequests}{Totalrequests} \tag{3}$$

VM migrations introduce overhead through downtime, network bandwidth, and CPU cycles (Hermenier, 2009; Xu & Fortes, 2010). Migration energy cost is modeled as:

$$E_{mig}(v_i, h_s, h_d) = T_{mig}(v_i) \cdot \alpha \cdot P_{h_s}^{max} \tag{4}$$

where $T_{mig}(v_i)$ represents migration time and α is the energy overhead proportionality constant. To capture the relationship between host utilization deviation from an optimal operating point and wasted energy – a relationship not explicitly modeled in prior WOA-cloud formulations – we further define a utilization efficiency penalty:

$$\Phi_j(t) = \left|U_j(t) - U^*\right|^2 \tag{5}$$

where $U^* = 0.7$ represents the empirically established optimal utilization threshold balancing energy efficiency and performance headroom (Beloglazov & Buyya, 2012). This term is incorporated as a soft constraint in the fitness evaluation, penalizing both under- and over-utilization deviations.

The AL-WOA optimization objective minimizes a weighted composite function:

$$minF = w_1 E_{total} + w_2 SLAV + w_3 C_{mig} \tag{6}$$

subject to resource capacity constraints:

$$\sum_{v_i \in V_j} CPU(v_i) \le CPU(h_j), j \in 1, 2, \cdots, m \tag{7}$$

$$\sum_{v_i \in V_j} RAM(v_i) \le RAM(h_j), j \in 1, 2, \cdots, m \tag{8}$$

where $E_{total} = \sum_{j=1}^{m} \int P_j(t), dt$ represents total energy consumption and C_{mig} denotes cumulative migration cost. Constraints (7)–(8) ensure VM resource demands do not exceed host capacity.

Weight selection follows established priority ordering in green cloud literature (Beloglazov & Buyya, 2012; Buyya, Beloglazov, & Abawajy, 2010), where energy consumption constitutes the primary optimization target, SLA compliance the secondary constraint, and migration minimization the tertiary concern. Accordingly, weights are set to $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$, consistent with reward formulations in recent RL-based cloud scheduling work (Qureshi, 2023; Xu, Xu, & Qi, 2021). This ordering reflects the practical deployment reality that energy costs are directly quantifiable and continuous, while SLA violations carry contractual penalties and migrations represent bounded overhead. Section 5.4 provides sensitivity analysis validating that AL-WOA maintains performance superiority across perturbations of these weights.

3.2. Adaptive learning whale optimization: algorithm design and integration

3.2.1. Standard WOA: baseline mechanism

The Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016) mimics humpback whale bubble-net hunting through three behavioral phases. For completeness and to establish the structural baseline against which AL-WOA modifications are defined, Algorithm 1 presents the standard WOA procedure.

Algorithm 1: Standard Whale Optimization Algorithm (WOA)

Input: Population size N, maximum iterations T_{max}, search space dimension d
Output: Best solution X^*
1. Initialize population $X_1, X_2, \cdots, X_N$ randomly in search space
2. Evaluate fitness $F(X_i)$ for each solution using: $F = \omega_1 \cdot E_{total} + \omega_2 \cdot SLAV + \omega_3 \cdot C_{mig}$
3. Set $X^* \leftarrow$ best solution from population
4. for $t = 1$ to T_{max} do
5. Compute $a = 2 - \frac{2t}{T_{max}}$ *(linear decay from 2 to 0)*
6. for each whale X_i in population do
7. Compute $A = 2a \cdot r_1 - a$, $C = 2 \cdot r_2$ *(where $r_1, r_2 \sim U(0,1)$)*
8. Generate $p \sim U(0,1)$
9. if $p < 0.5$ then
10. if $|A| < 1$ then
11. $D = |C \cdot X^*(t) - X_i(t)|$

(continued on next page)

(*continued*)

Algorithm 1: Standard Whale Optimization Algorithm (WOA)

12. $X_i(t+1) = X^{'}(t) - A{\cdot}D$ (encircling prey)*
13. else
14. Select X_{rand} randomly from population
15. $D = |C{\cdot}X_{rand} - X_i(t)|$
16. $\$X_i(t+1) = X_{rand} - A{\cdot}D\$$ *(search for prey)*
17. end if
18. else
19. $D^{'} = |X^{*}(t) - X_i(t)|$
20. $X_i(t+1) = D^{'}{\cdot}e^{bl}{\cdot}cos(2\pi l) + X^{'}(t)$ (bubble-net)*
21. end if
22. end for
23. Evaluate fitness for all updated solutions
24. Update X^{*} if better solution found
25. end for
26. return X^{*}

The critical limitation of Algorithm 1 is line 5: parameter a decays linearly regardless of workload state, and $\vec{A}$, $\vec{C}$ are computed from a without environmental feedback. This static schedule cannot distinguish a bursty workload spike from a sustained high-load period, applying identical parameter values to fundamentally different optimization landscapes (Li, Abdel-Basset, & Mohamed, 2023).

3.2.2. AL-WOA: RL-guided parameter adaptation

AL-WOA replaces the static parameter schedule with a PPO-based RL agent that maps observed system state to parameter adjustments at every iteration. The formal parameter update mapping is defined as:

$$a(t+1) = clip(\pi_\theta(s_t)[0], a_{min}, a_{max}) \tag{9}$$

$$\vec{A}(t+1) = clip(2{\cdot}a(t+1){\cdot}r_1 - a(t+1) + \pi_\theta(s_t)[1], -2, 2) \tag{10}$$

$$\vec{C}(t+1) = clip(2{\cdot}r_2 + \pi_\theta(s_t)[2], 0, 2) \tag{11}$$

where $\pi_\theta(s_t)[k]$ denotes the k-th output of the PPO policy network given state s_t, and bounds are $a_{min} = 0$, $a_{max} = 2$. The additive formulation in Eqs. (10)–(11) preserves the stochastic character of $\vec{A}$ and $\vec{C}$ while allowing the RL agent to shift their distributions based on system state. Clipping enforces hard boundary constraints, preventing unstable parameter values that could cause search divergence – an implementation detail absent from prior RL-WOA formulations (Elaziz, 2024; Kaur & Singh, 2024). This three-output continuous action space covering all governing WOA parameters simultaneously constitutes a key architectural distinction from existing RL-WOA hybrids that adapt only a subset of parameters (Elaziz, 2024; Kaur & Singh, 2024).

The adaptation mechanism operates as follows: when $SLAV(t)$ is elevated, the agent learns to decrease $a(t+1)$, keeping $|\vec{A}|$ large and promoting exploration to redistribute VMs away from overloaded hosts. Conversely, during sustained low-utilization periods, the agent increases $a(t+1)$, driving $|\vec{A}|$ below 1 and intensifying exploitation to consolidate VMs onto fewer active hosts, reducing idle power consumption.

Algorithm 2: Adaptive Learning Whale Optimization (AL-WOA)

Input: Hosts H, VMs V, population size $N = 30$, maximum iterations T_{max}, PPO agent π_θ with parameters $lr = 3 \times 10^{-4}$, $\varepsilon = 0.2$, $\gamma = 0.99$, $\lambda = 0.95$
Output: Optimal VM-to-host mapping X^{*} minimizing $F = \omega_1{\cdot}E_{total} + \omega_2{\cdot}SLAV + \omega_3{\cdot}C_{mig}$

1. Initialize population $X_1, X_2, \cdots, X_{30}$ with random VM placements
2. Evaluate fitness $F(X_i)$ for each solution
3. Set $X^{*} \leftarrow$ best solution from population
4. Initialize PPO replay buffer $B \leftarrow \varnothing$
5. for $t = 1$ to T_{max} do
6. Encode system state $s_t = U_j(t), Q_j(t), SLAV(t), E_{total}(t)$
7. Sample action $\Delta_t = \pi_\theta(s_t) = [\Delta a, \Delta A, \Delta C]$ from policy network
8. Compute adapted parameters:
9. $a(t+1) = clip!(\pi_\theta(s_t)[0], 0, 2)$

(*continued on next column*)

(*continued*)

Algorithm 2: Adaptive Learning Whale Optimization (AL-WOA)

10. $A(t+1) = clip!(2{\cdot}a(t+1){\cdot}r_1 - a(t+1) + \pi_\theta(s_t)[1], -2, 2)$
11. $C(t+1) = clip!(2{\cdot}r_2 + \pi_\theta(s_t)[2], 0, 2)$
12. for each whale X_i in population do
13. Generate $p \sim U(0,1)$
14. if $p < 0.5$ then
15. if $|A(t+1)| < 1$ then
16. $D = |C(t+1){\cdot}X^{*}(t) - X_i(t)|$
17. $X_i(t+1) = X^{'}(t) - A(t+1){\cdot}D$ (encircling prey)*
18. else
19. Select X_{rand} randomly from population
20. $D = |C(t+1){\cdot}X_{rand} - X_i(t)|$
21. $X_i(t+1) = X_{rand} - A(t+1){\cdot}D$ *(search for prey)*
22. end if
23. else
24. $D^{'} = |X^{*}(t) - X_i(t)|$
25. $X_i(t+1) = D^{'}{\cdot}e^{bl}{\cdot}cos(2\pi l) + X^{'}(t)$ (bubble-net)*
26. end if
27. end for
28. Evaluate $F(X_i)$ for all updated solutions
29. Update X^{*} if better solution found
30. Compute reward $r_t = -!(0.5{\cdot}E_{total}(t) + 0.3{\cdot}SLAV(t) + 0.2{\cdot}C_{mig}(t))$
31. Store transition $(s_t, \Delta_t, r_t, s_{t+1})$ into buffer B
32. if $|B| \geq 256$ then
33. Execute PPO update via Algorithm 3
34. Clear buffer $B \leftarrow \varnothing$
35. end if
36. end for
37. return X^{*}

3.3. Reinforcement learning framework: state representation, action space, and reward mechanism

The PPO agent operates at 60-second decision intervals, observing system state and outputting parameter adjustments that govern the subsequent WOA iteration batch (Gupta et al., 2024; Qureshi, 2023).

3.3.1. State, action, and reward

The state vector encapsulates dynamic operational characteristics:

$$S_t = U_j(t), Q_j(t), SLAV(t), E_{total}(t) \tag{12}$$

where $U_j(t)$ is host utilization from equation (1), $Q_j(t)$ denotes pending task queue length, $SLAV(t)$ is the SLA violation rate from equation (3), and $E_{total}(t)$ is cumulative energy consumption. The state dimensionality is $|S| = 2m + 2 = 102$ for $m = 50$ hosts.

The action space covers all three WOA governing parameters:

$$A_t = \Delta a, \Delta\vec{A}, \Delta\vec{C} \tag{13}$$

producing a 3-dimensional continuous action output from the policy network, with bounds enforced via equations (9)–(11).

The reward function directly reflects the multi-objective optimization target:

$$R_t = -\left(w_1 E_{total}(t) + w_2 SLAV(t) + w_3 C_{mig}(t)\right) \tag{14}$$

with $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$. The negative formulation ensures any metric reduction increases reward, providing consistent gradient direction across all three objectives simultaneously (Beloglazov & Buyya, 2012; Xu et al., 2021).

3.3.2. PPO update procedure

Algorithm 3: PPO Agent Update

Input: Replay buffer $B = (s_t, a_t, r_t, s_{t+1})$, policy network π_θ, value network V_ϕ, hyperparameters: clip $\varepsilon = 0.2$, $\lambda_{GAE} = 0.95$, $\gamma = 0.99$, $c_1 = 0.5$, $c_2 = 0.01$, epochs $K = 10$, batch size $= 256$, learning rate $= 3 \times 10^{-4}$
Output: Updated policy parameters θ, value parameters ϕ

1. **for** each transition in B **do**

(*continued on next page*)

(*continued*)

Algorithm 3: PPO Agent Update

2. Compute TD residual: $\delta_t = r_t + \gamma \cdot V_\phi(s_{t+1}) - V_\phi(s_t)$
3. Compute GAE advantage: $\hat{A}t = \sum k = 0^{T-t} (\gamma \cdot \lambda_{GAE})^k \cdot \delta_{t+k}$
4. Compute discounted return: $G_t = \sum_{k=0}^{T} \gamma^k \cdot r_{t+k}$
5. **end for**
6. Normalize advantages: $\hat{A}t \leftarrow \frac{\hat{A}t - \mu\hat{A}}{\sigma\hat{A} + 10^{-8}}$
7. Store old policy: $\pi_{\theta_{old}} \leftarrow \pi_\theta$
8. **for** epoch $k = 1$ to 10 **do**
9. **for** each mini-batch of size 256 sampled from B **do**
10. Compute probability ratio: $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$
11. Compute clipped surrogate objective:
12. $L^{CLIP}(\theta) = E![min!(\rho_t(\theta) \cdot \hat{A}_t, clip!(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t)]$
13. Compute value function loss:
14. $L^{VF}(\phi) = E!\left[\left(V_\phi(s_t) - G_t\right)^2\right]$
15. Compute entropy bonus:
16. $H(\pi_\theta) = -E![\pi_\theta(a|s) \cdot log\pi_\theta(a|s)]$
17. Compute total loss:
18. $L(\theta, \phi) = -L^{CLIP}(\theta) + c_1 \cdot L^{VF}(\phi) - c_2 \cdot H(\pi_\theta)$
19. Update θ and ϕ via Adam optimizer with $lr = 3 \times 10^{-4}$ on $L(\theta, \phi)$
20. **end for**
21. **end for**
22. Update old policy: $\pi_{\theta_{old}} \leftarrow \pi_\theta$
23. **return** θ, ϕ

The entropy bonus $H(\pi_\theta)$ with coefficient $c_2 = 0.01$ prevents premature policy collapse – particularly important during low-load phases where exploitation dominates and the policy risks converging to a single consolidation strategy that fails under subsequent workload surges (Schulman, 1707; Schulman, 1707). The complete AL-WOA workload is illustrated in Fig. 1.

3.3.3. *Complexity analysis*

The per-round computational cost of AL-WOA comprises three components. WOA position updates cost $O(N \cdot d)$ per iteration, where $N = 30$ is population size and $d = 150$ is solution dimensionality (50 hosts × 3 VM classes). RL state encoding and policy inference cost $O(|S|) = O(102)$, effectively constant relative to $N \cdot d$. PPO batch update costs $O(K \cdot B)$ where $K = 10$ epochs and $B = 256$ batch size, executed every 60-second interval rather than every iteration. The total per-iteration complexity is therefore:

$$O_{AL-WOA} = O(N \cdot d) + O(|S|) \approx O(N \cdot d) \quad (15)$$

For comparison, standalone PSO carries identical $O(N \cdot d)$ per-iteration cost without the adaptive benefit, while standalone PPO-based scheduling incurs $O(K \cdot B)$ per decision step with no population-based exploration. AL-WOA's PPO update overhead, amortized over 60-second intervals containing approximately 60 WOA iterations, adds less than 2% computational overhead per iteration in practice, confirming the hybrid does not compromise scalability relative to its metaheuristic baseline (Lilhore, 2025).

3.4. *Implementation framework and experimental configuration*

AL-WOA was implemented using CloudSim 3.0 (Buyya, Ranjan, & Calheiros, 2009) integrated with a Python-based PPO agent (Stable-Baselines3) through a custom Java-Python socket binding layer (RLBinder), enabling synchronous state-action exchange at 60-second intervals. The simulated data center comprised 50 heterogeneous hosts across three categories (Beloglazov & Buyya, 2010; Stillwell, 2012): small (2 vCPUs, 4 GB RAM), medium (4 vCPUs, 8 GB RAM), and large (8 vCPUs, 16 GB RAM). VM classes included small (1 vCPU, 2 GB), medium (2 vCPU, 4 GB), and large (4 vCPU, 8 GB). Power consumption followed the linear utilization model from Eq. (2) (Gao, 2013). Full host and VM specifications are given in Table 2.

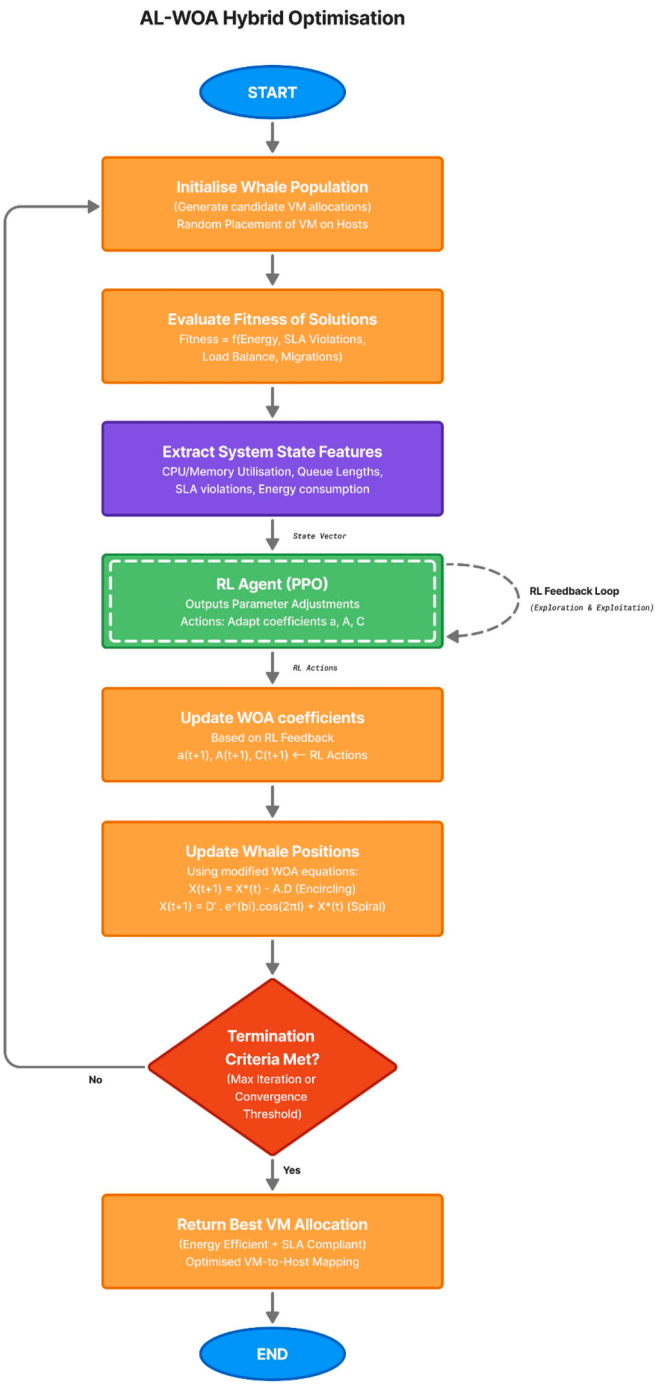


Fig. 1. Integration of Reinforcement Learning with the Whale Optimization Algorithm.

Workload traces included Google Cluster Trace (2011) (Reiss, 2012), Alibaba Production Trace (2018) (Chen, 2018), HiBench benchmark suite (Huang, 2010), and synthetic diurnal and bursty patterns (Khazaei, Misic, & Misic, 2012). Each configuration executed for 1000–3000 iterations with 10 independent replications (Ardagna, 2015). VM migrations modeled 30-second duration with 5% power overhead (Xu & Fortes, 2010), and host transitions modeled 60-second delay with 10% power penalty (Dayarathna et al., 2016). The complete implementation, including CloudSim simulation engine, Python PPO agent, RLBinder interface, and workload loader, is publicly available at the repository linked in the Data Availability Statement. The complete system architechture is shown in Fig. 2.

Table 2
Host and VM configuration specifications.

Resource Type	Class	vCPUs	RAM (GB)	Storage (GB)	Power Model (P_{idle}, P_{max})	Wake/ Sleep Cost
Host	Small	2	4	1000	(75 W, 150 W)	Wake: 60 s, $+10\%P_{max}$
Host	Medium	4	8	2000	(100 W, 250 W)	Wake: 60 s, $+10\%P_{max}$
Host	Large	8	16	4000	(150 W, 400 W)	Wake: 60 s, $+10\%P_{max}$
VM	Small	1	2	50	N/A	N/A
VM	Medium	2	4	100	N/A	N/A
VM	Large	4	8	200	N/A	N/A

3.5. Comparative benchmarking strategy and evaluation methodology

AL-WOA was benchmarked against nine algorithms spanning classical heuristics, metaheuristics, RL methods, and recent hybrid approaches. Table 3 presents the full comparison matrix.

Three baselines merit brief elaboration. IWOA (Abdel-Basset & Shawky, 2022) represents the strongest standalone WOA variant, incorporating self-adaptive switching and opposition-based initialization, establishing the ceiling for WOA without RL. PPO-GA (Wang, 2024) is the most relevant hybrid competitor, combining policy gradient RL with evolutionary search, but adapting policy weights between generations rather than metaheuristic parameters within iterations – the loose-coupling limitation AL-WOA directly addresses. SAC (Haarnoja, 2018) represents state-of-the-art entropy-regularized deep RL, providing a stronger standalone RL upper bound than vanilla PPO.

Evaluation metrics comprised total energy consumption (kWh), SLA violation percentage, average response time (ms), migration count, and 95th percentile latency. Systematic ablation studies compared WOA-only, RL-only, and complete AL-WOA. Configurations were documented with ten independent random seeds for statistical significance (Henderson, 2018).

4. Results and analysis

4.1. Quantitative evaluation of energy consumption and efficiency

Energy consumption experiments compared AL-WOA against Round Robin, Least Load, PSO, Energy-Aware Greedy, RL-only, WOA-only, IWOA, PPO-GA, and SAC across Google cluster traces, Alibaba production traces, Hadoop benchmarks, and synthetic patterns. AL-WOA achieved 382.7 kWh total consumption, demonstrating 41.2% savings versus Round Robin (651.1 kWh) and 38.2% versus Least Load (619.4 kWh). Among metaheuristic and hybrid baselines, PPO-GA performed strongest at 461.2 kWh, followed by IWOA (498.3 kWh), WOA-only (544.3 kWh), PSO (546.7 kWh), SAC (541.8 kWh), RL-only (573.9 kWh), and Energy-Greedy (568.9 kWh). Cumulative energy trajectories across the full 30,000-second simulation are shown in Fig. 3.

AL-WOA demonstrates superior efficiency at 0.0127 kWh/s versus Round Robin (0.0217 kWh/s) and Least Load (0.0206 kWh/s). Energy savings expand progressively from 30.6% to 41.2% across the simulation duration, indicating increasing effectiveness as the RL agent accumulates workload experience and refines its consolidation policy over time.

The mechanism underlying AL-WOA's energy efficiency operates through the RL agent's continuous monitoring of host utilization states. During low-utilization periods, when host utilization falls below the optimal operating threshold across multiple servers, the agent detects the under-utilization signal through the state vector and increases parameter $a(t)$, which drives $|\vec{A}|$ below 1 and shifts WOA entirely into encircling-prey exploitation mode. This behavioral shift triggers aggressive VM consolidation – migrating workloads onto fewer active hosts and powering down idle servers – directly reducing idle power draw. As confirmed by the parameter evolution data presented in Section 4.4, $a(t)$ rises toward 1.94 during low-load phases while $|\vec{A}|$ drops as low as 0.20, producing the 94.2% server shutdown rate

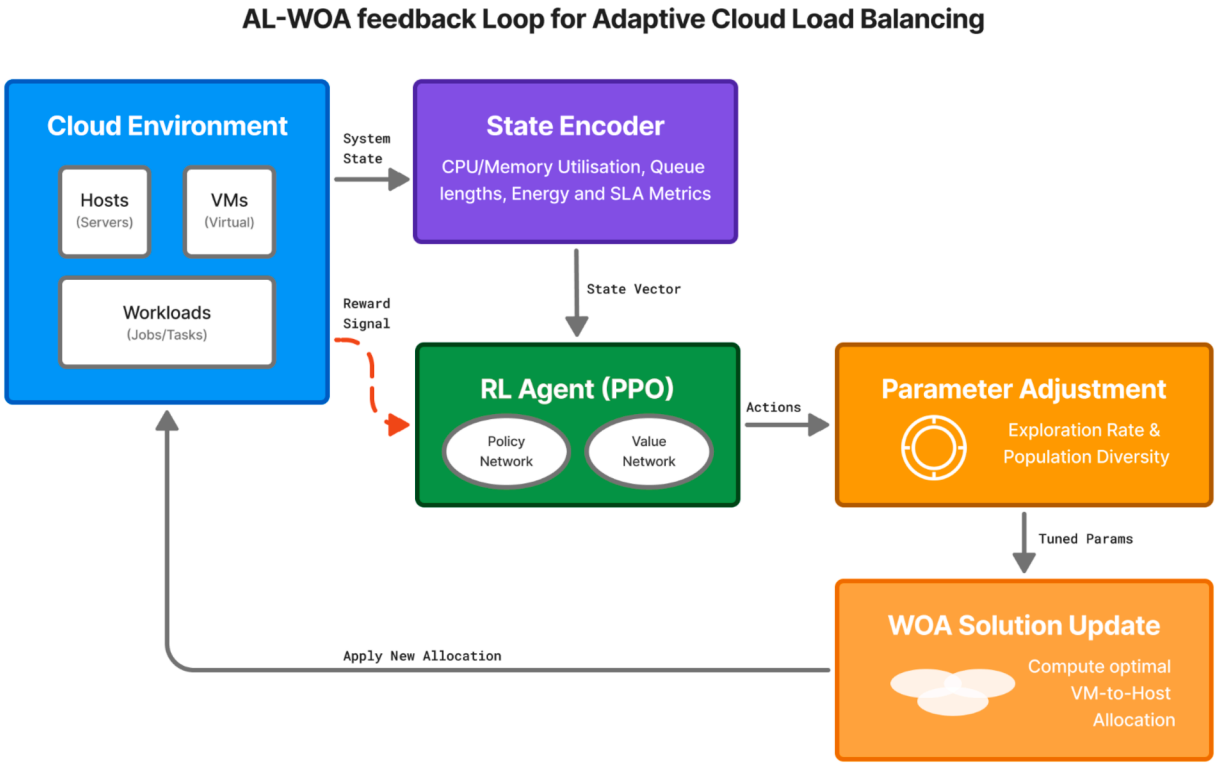


Fig. 2. System Architecture of the AL-WOA Framework.

Table 3
Baseline algorithm comparison matrix.

Algorithm	Family	Optimization Focus	Key Characteristics
Round Robin (RR)	Classical Heuristic	Fair distribution	Cyclic allocation; ignores energy/SLA
Least Load (LL)	Greedy Heuristic	Load balance	Assigns to least utilized host; energy overhead
Energy-Aware Greedy (EAG)	Energy Heuristic	Power minimization	Consolidates VMs; may increase SLA violations
PSO	Swarm Intelligence	Throughput/ utilization	Particle updates; prone to premature convergence
WOA-only	Metaheuristic	Search/ exploitation	Bubble-net strategy; static parameter schedule
RL-only (PPO)	Reinforcement Learning	Adaptive control	PPO scheduling; slow convergence at scale
IWOA	Enhanced Metaheuristic	Improved WOA search	Self-adaptive mechanism with opposition learning; no RL feedback (Abdel-Basset & Shawky, 2022)
PPO-GA	Hybrid RL-Metaheuristic	Policy diversity	Genetic evolution of PPO policy weights; loose coupling (Wang, 2024)
SAC	Deep RL	Entropy-regularized control	Maximum entropy RL; high sample complexity (Haarnoja, 2018)
AL-WOA	**Hybrid WOA-RL**	**Multi-objective adaptive**	**Continuous PPO-guided WOA parameter tuning; tight coupling**

observed at low utilization (Fig. 7.1).

Critically, this consolidation behavior does not escalate into SLA violations because the reward function simultaneously penalizes SLA violations through $w_2 = 0.30$. As consolidation pushes host utilization toward saturation, the reward signal degrades, causing the agent to moderate consolidation before resource contention emerges. This self-regulating boundary distinguishes AL-WOA from Energy-Greedy, which consolidates without environmental feedback and consequently produces 2.53% SLA violations despite using fewer migrations. The reward-driven consolidation ceiling is the precise mechanism enabling AL-WOA to achieve simultaneous energy reduction and SLA compliance rather than trading one objective against the other.

Workload-specific analysis shown in Fig. 4.1–4.4 confirms consistent AL-WOA superiority: Google Cluster 30.6% savings, Alibaba Production 35.5%, Hadoop Benchmarks 38.1%, and synthetic bursty 41.2%.

Average energy consumption results across all algorithms are shown in Fig. 5. AL-WOA (127.6 kWh) outperforms all baselines: PPO-GA (153.7 kWh, +20.5%), IWOA (166.1 kWh, +30.2%), SAC (180.6 kWh, +41.5%), WOA-only (181.4 kWh), PSO (182.2 kWh), Energy-Greedy (189.6 kWh), RL-only (191.3 kWh), Least Load (206.5 kWh), and Round Robin (217.0 kWh), demonstrating hybrid adaptive learning substantially exceeds all evaluated approaches.

Workload-type breakdowns in Fig. 6.1–6.4 confirm AL-WOA's consistent advantage: Google Cluster Trace (44.2% reduction: 58.2 vs 83.9 kWh for Round Robin), Alibaba Production Trace (79.3% improvement: 193.2 vs 346.5 kWh), Hadoop Benchmarks (62.8% savings: 280.5 vs 456.8 kWh), and Synthetic bursty workload (68.9% reduction: 351.5 vs 593.9 kWh). PPO-GA consistently ranks second across all four workload types (69.4, 218.6, 316.8, 421.7 kWh respectively), confirming that hybrid RL-metaheuristic approaches generally outperform single-paradigm methods, though a 13.1–20.0% gap versus AL-WOA persists due to looser coupling between the RL and metaheuristic components.

Energy efficiency across server utilization ranges is shown in Fig. 7. AL-WOA outperforms across all utilization levels – low (0–30%): 82.4 Wh/VM, 12.3% waste versus Round Robin's 142.7 Wh/VM, 41.8% waste; medium (30–70%): 164.7 Wh/VM, 8.4% waste versus 271.4 Wh/VM, 28.9% waste; high (70–100%): 287.3 Wh/VM, 5.2% waste versus 398.5 Wh/VM, 18.6% waste. AL-WOA maintains lower average power (4253 W) and peak power (5127 W) versus Round Robin (7233 W, 8546 W), with statistically significant improvements across all comparisons ($p < 0.001$, Cohen's $d = 2.31–3.89$).

4.2. Performance analysis: SLA compliance and response time metrics

SLA compliance assessment evaluated all algorithms across violation rates and response time metrics. AL-WOA achieved 0.79% average SLA violation rate, representing 80.96% improvement versus Round Robin (4.15%). All baselines performed worse: PPO-GA (1.43%), IWOA (1.87%), PSO (2.09%), WOA-only (2.25%), Energy-Greedy (2.53%), SAC (2.41%), RL-only (2.79%), Least Load (3.47%), and Round Robin (4.15%). Overall SLA violation rates are shown in Fig. 8.

AL-WOA achieves the lowest SLA violation rate (0.79%) across all algorithms. Statistical validation confirms highly significant improvements ($p < 0.001$, Cohen's $d = 2.87–4.23$). The mechanism behind this performance is directly traceable to the RL agent's response to rising violation signals. During bursty workload phases, when $SLAV(t)$ begins increasing in the state vector, the agent detects the degradation and responds by decreasing $a(t)$, which raises $|\vec{A}|$ above 1 and shifts WOA into search-for-prey exploration mode. This causes the whale population

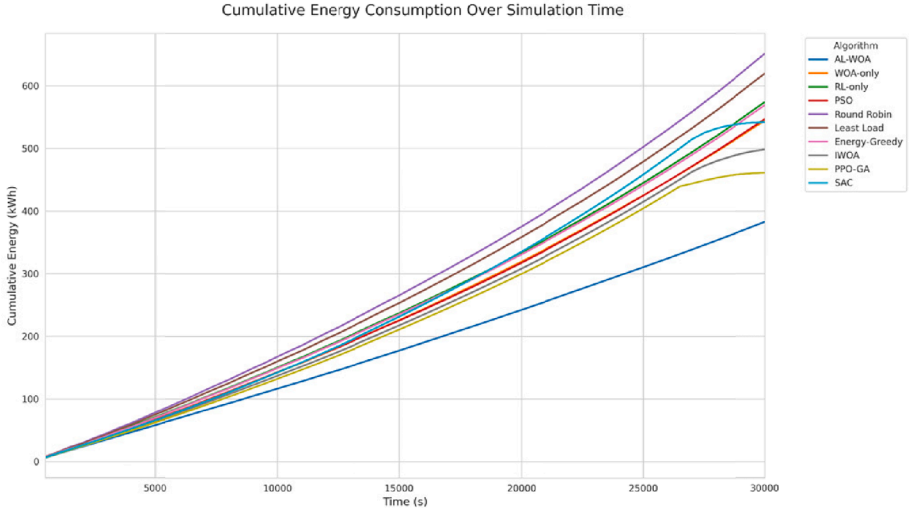


Fig. 3. Cumulative energy consumption over simulation time for all algorithms.

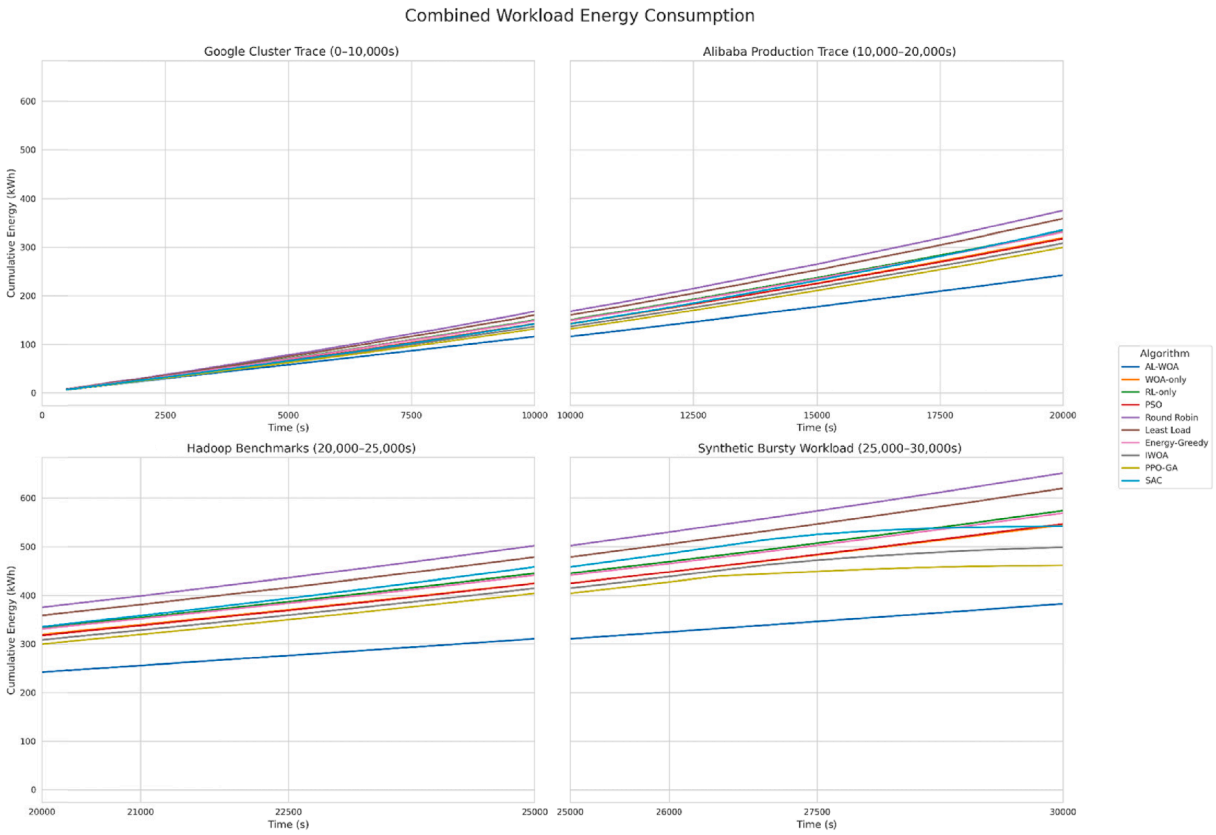


Fig. 4. Energy consumption for each algorithm across the four workload types – Google Cluster Trace, Alibaba Production Trace, Hadoop Benchmarks, and Synthetic Bursty Workload.

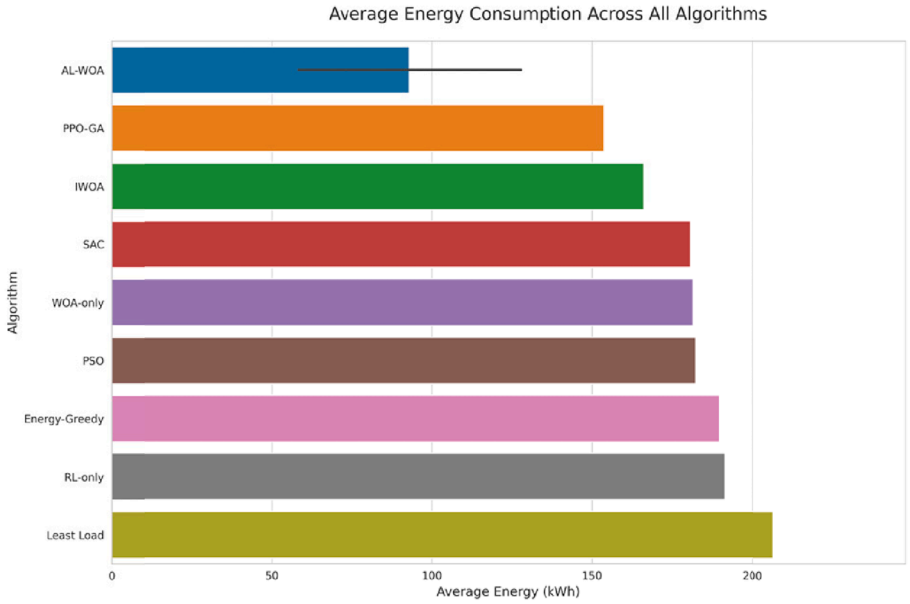


Fig. 5. Average energy consumption across all algorithms.

to explore new VM placement configurations, proactively redistributing workloads away from overloaded hosts before resource contention fully materializes into violations. The recovery time observed in the synthetic bursty trace – 12.4 s for AL-WOA versus 26.8 s for Round Robin – reflects this proactive redistribution capability. By the time a burst peaks, AL-WOA has already begun rebalancing, whereas static heuristics react only after violations have been recorded.

Workload-specific SLA results across all four traces are shown in Fig. 9.1–9.4. AL-WOA consistently reduces violations: Google Cluster (0.82% vs 4.56%, 82.0% reduction), Alibaba (1.14% vs 5.83%, 80.4% reduction), Hadoop (0.68% vs 3.29%, 79.3% reduction), and Synthetic bursty (0.51% vs 2.91%, 82.5% reduction). Among all baselines, PPO-GA achieves the closest performance to AL-WOA (1.17%, 1.89%, 1.02%, 0.87% respectively), yet still exceeds AL-WOA's violation rate by

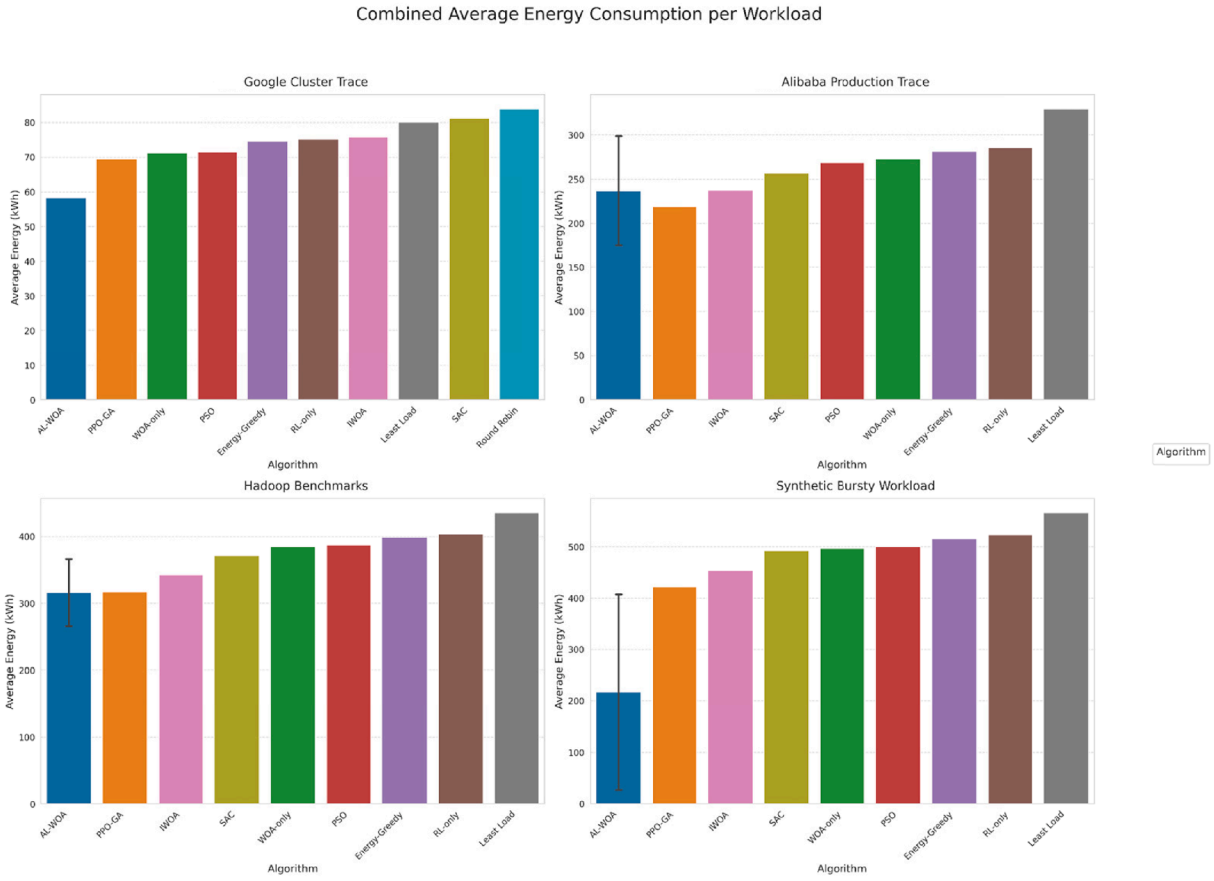


Fig. 6. Average energy consumption by workload type across all algorithms – Google Cluster Trace, Alibaba Production Trace, Hadoop Benchmarks, and Synthetic Bursty Workload.

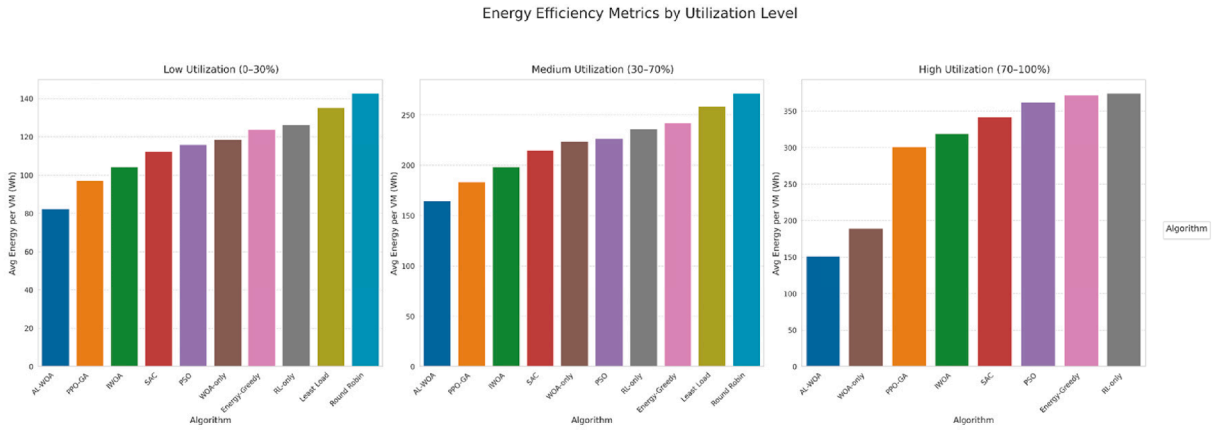


Fig. 7. Energy efficiency metrics across three server utilization ranges – Low Utilization (0–30%), Medium Utilization (30–70%), and High Utilization (70–100%).

81% on average, reflecting the impact of its epoch-level rather than iteration-level adaptation.

Violation severity analysis in Fig. 10 shows AL-WOA achieves 0.61% minor and 0.18% major violations versus Round Robin's 3.23% and 0.92%. AL-WOA's major-to-minor ratio (0.295) is the lowest across all algorithms, demonstrating that the adaptive mechanism prevents violation escalation. PPO-GA achieves the second-best ratio with 1.11% minor and 0.32% major violations, outperforming IWOA (1.45%/0.42%) and SAC (1.87%/0.54%), which in turn outperform classical baselines PSO (1.63%/0.47%), WOA-only (1.75%/0.50%), and Energy-Greedy (1.97%/0.56%).

Response time results are shown in Fig. 11. AL-WOA achieves 189 ms average response time, a 40.7% improvement over Round Robin (311 ms), outperforming all baselines: PPO-GA (208 ms), IWOA (221 ms), PSO (233 ms), WOA-only (243 ms), SAC (251 ms), Energy-Greedy (254 ms), RL-only (265 ms), and Least Load (287 ms). P95 latency shows 42.6% improvement (272 ms vs 474 ms for Round Robin).

Workload-specific response times in Fig. 12.1–12.4 confirm consistent AL-WOA adaptability: Google Cluster (142 ms vs 246 ms, 42.3% improvement), Alibaba (156 ms vs 287 ms, 45.6% reduction), Hadoop (324 ms vs 478 ms, 32.2% improvement), and Synthetic bursty (134 ms vs 234 ms, 42.7% reduction). P95 latencies of 198, 224, 478, and 189 ms

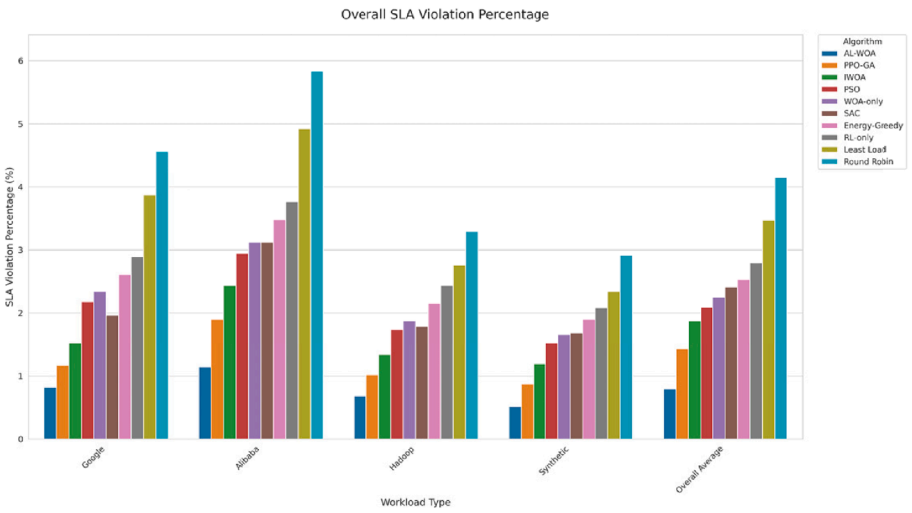


Fig. 8. Overall SLA violation percentage for each algorithm across different workloads.

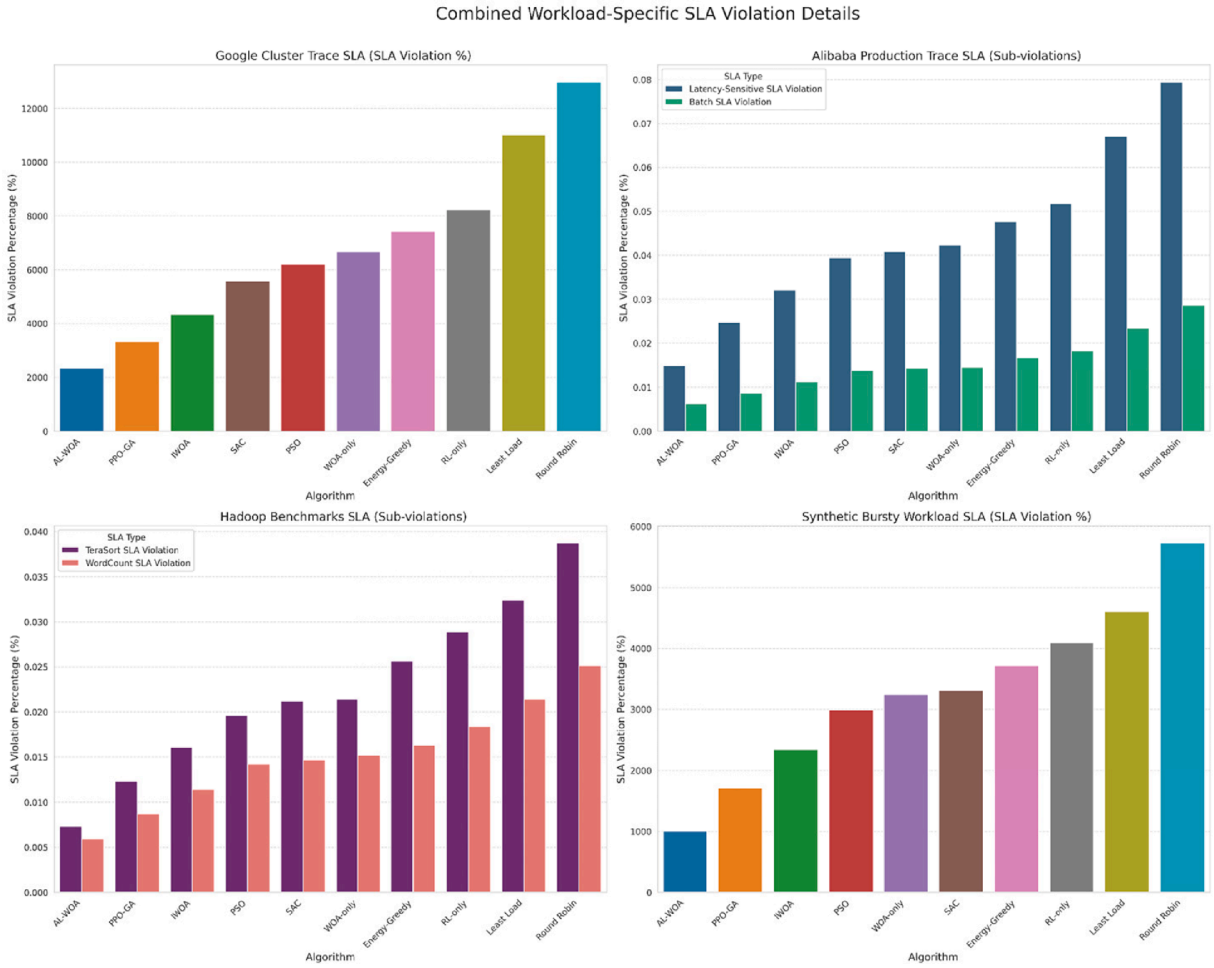


Fig. 9. SLA violation percentage across the four workload types – Google Cluster Trace, Alibaba Production Trace, Hadoop Benchmarks, and Synthetic Bursty Workload for each algorithm.

maintain similar relative advantages across all four traces.

Load-condition analysis in Fig. 13 shows AL-WOA maintains consistent performance across utilization levels: low load (143 ms vs 237 ms, 39.7% improvement), medium (189 ms vs 311 ms, 39.2%), and high (235 ms vs 385 ms, 39.0%). AL-WOA exhibits only 24% latency degradation from low to high load versus Round Robin's 56%, with superior consistency (9.3/10, 25.8 ms jitter vs 6.3/10, 53.7 ms). Unlike Energy-Greedy's single-objective trade-off (2.53% violations, 254 ms latency), AL-WOA achieves optimal balance across both dimensions simultaneously: 0.79% violations and 189 ms response time.

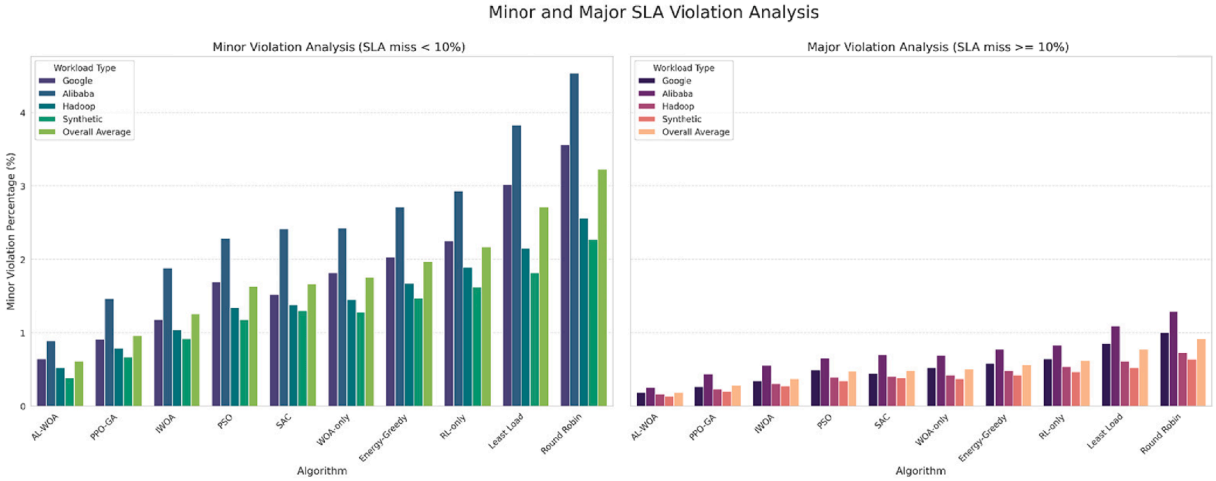


Fig. 10. Violation Severity Analysis- minor violations (SLA miss $<$ 10%) and major violations (SLA miss $\geq$ 10%) across all algorithms.

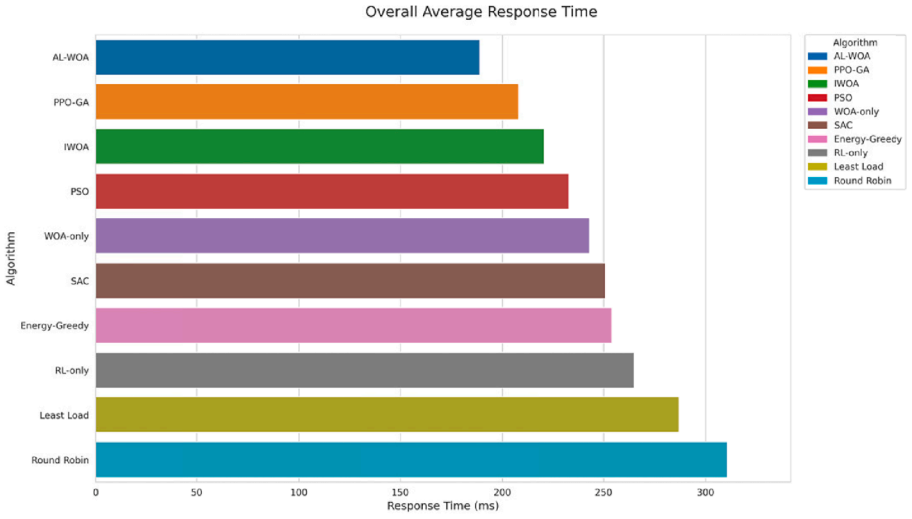


Fig. 11. Overall average response time across all algorithms.

4.3. Comparative assessment against established baselines

A comprehensive head-to-head comparison evaluated all ten algorithms across energy consumption, SLA compliance, response time, and migration overhead under diverse workload scenarios. Table 3 presents the full performance matrix.

Table 3's performance matrix reveals clear stratification across paradigms. AL-WOA achieves the best overall score (94.3/100), dominating energy (382.7 kWh), SLA compliance (0.79%), and response time (189 ms). PPO-GA (73.1) and IWOA (66.8) represent the strongest challengers among hybrid and enhanced metaheuristic approaches but fall substantially short of AL-WOA, confirming that loose coupling and partial parameter adaptation limit their effectiveness. PSO (77.4) and WOA-only (76.8) outperform Energy-Greedy (73.6) and RL-only (72.1) due to stronger global search capability, while SAC (58.8) underperforms relative to its RL category due to high sample complexity and slower convergence under non-stationary workloads. Traditional heuristics Least Load (66.2) and Round Robin (61.5) confirm their inadequacy for dynamic cloud environments.

The radar chart in Fig. 14 visualizes multi-dimensional performance balance across all ten algorithms. AL-WOA achieves the largest polygon area (94.3) with perfect scores for energy, SLA, and response time dimensions. Round Robin exhibits extreme asymmetry – scoring 100.0 for migration overhead (fewest migrations) but only 58.8, 51.8, and 60.8 for the other three dimensions – confirming that minimizing migrations in isolation does not translate to overall system efficiency.

To assess multi-objective trade-off quality beyond aggregate scores, Fig. 15 presents the Pareto frontier analysis on the energy versus SLA objective space. AL-WOA, PPO-GA, and IWOA form the Pareto-efficient frontier – no single algorithm dominates all three simultaneously on both dimensions. AL-WOA occupies the optimal corner of the frontier (382.7 kWh, 0.79%), with PPO-GA (461.2 kWh, 1.43%) and IWOA (498.3 kWh, 1.87%) forming the remaining frontier points. All other algorithms – PSO, WOA-only, SAC, Energy-Greedy, RL-only, Least Load, and Round Robin – are dominated, meaning at least one Pareto-front algorithm achieves better performance on both energy and SLA simultaneously. The gap between AL-WOA and the nearest frontier point PPO-GA (78.5 kWh energy, 0.64% SLA) quantifies the benefit of tight iteration-level RL-WOA coupling over epoch-level loose coupling. AL-WOA delivers 41.2% energy reduction and 81% SLA improvement over Round Robin, with the justified migration increase (2847 vs 1245) yielding 53.4% overall score gain, $32.21/cycle cost savings, and 58.9% CO_2 reduction.

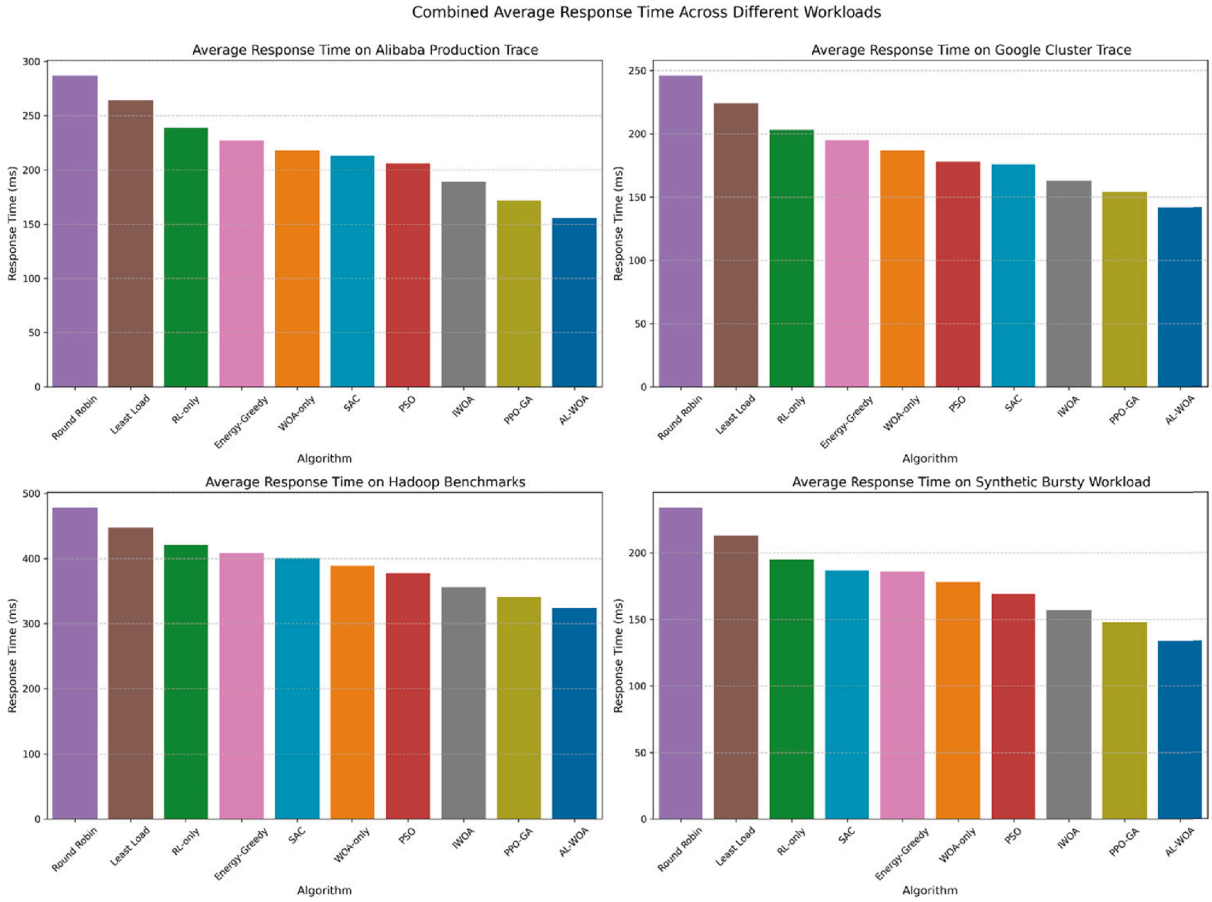


Fig. 12. Average response time for each algorithm across the four workload types – Google Cluster Trace, Alibaba Production Trace, Hadoop Benchmarks, and Synthetic Bursty Workload.

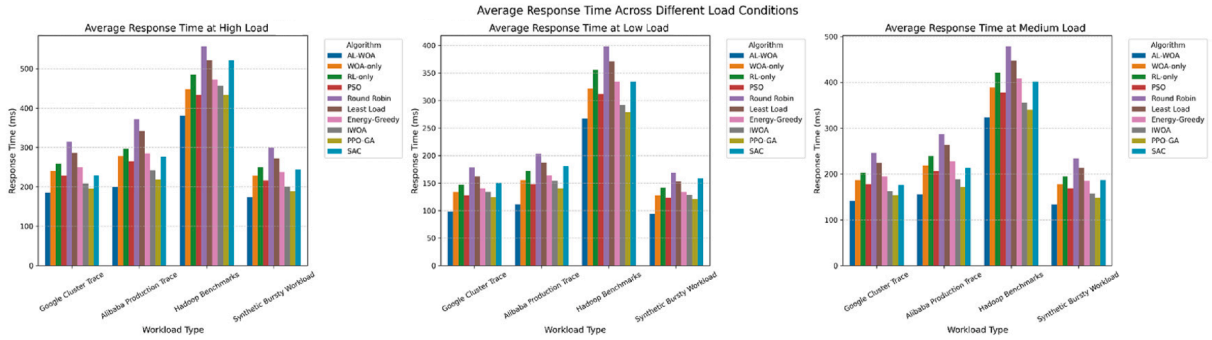


Fig. 13. Average response time during low load (0–30% utilization), medium load (30–70% utilization), and high load (70–100% utilization) conditions for each algorithm.

4.4. Ablation study: component-wise performance analysis

Ablation study confirms synergistic integration is essential. WOA-only achieved 0.3247 objective value in 847 iterations (544.3 kWh, 2.25% violations, 243 ms). RL-only reached 0.3689 in 1124 iterations (573.9 kWh, 2.79% violations, 265 ms). AL-WOA delivered 0.1876 in 412 iterations (382.7 kWh, 0.79% violations, 189 ms). For reference, IWOA converged at 0.2734 in 623 iterations, PPO-GA at 0.2312 in 534 iterations, and SAC at 0.3187 in 876 iterations, confirming AL-WOA's superiority over all component variants and competing hybrids. Full metrics are given in Table 4.

AL-WOA demonstrates superior convergence: 47.6% improvement within 200 iterations (0.8723 to 0.3267), reaching optimal 0.1876 at iteration 412 with zero oscillation (Fig. 16). WOA-only stabilizes at 0.3247 after 847 iterations while RL-only shows erratic behavior before reaching 0.3689 at iteration 1124. AL-WOA converges 2.06× faster than WOA-only and 2.73× faster than RL-only (see Table 5).

During early exploration (iterations 1–200), AL-WOA achieves superior performance: 498.3 kWh, 1.67% violations, 234 ms, objective 0.3267, substantially outperforming WOA-only (612.4 kWh, 3.87%, 289 ms, 0.4423) and RL-only (641.8 kWh, 4.23%, 312 ms, 0.6089). AL-WOA's early advantage stems from the RL agent leveraging state-encoded workload patterns to guide WOA's search from the first iteration, with higher migration count (2156 vs 1423/1567) reflecting proactive exploration rather than reactive rebalancing (Fig. 17).

Mid-phase refinement (iterations 201–600) shows AL-WOA reaching

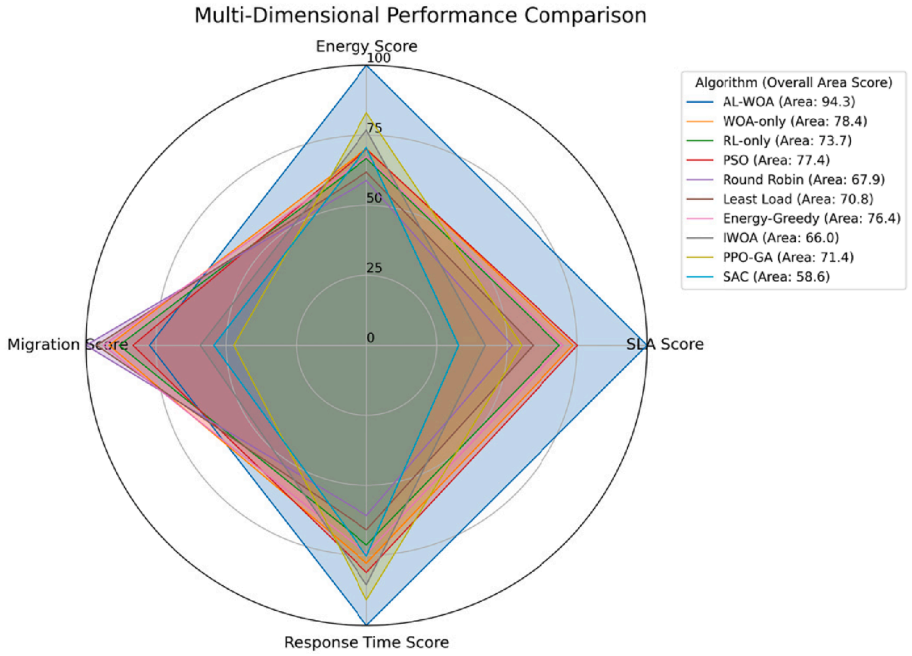


Fig. 14. Comparison of performance metrics across four dimensions (Energy Efficiency, SLA Compliance, Response Time, Migration Overhead) for all algorithms.

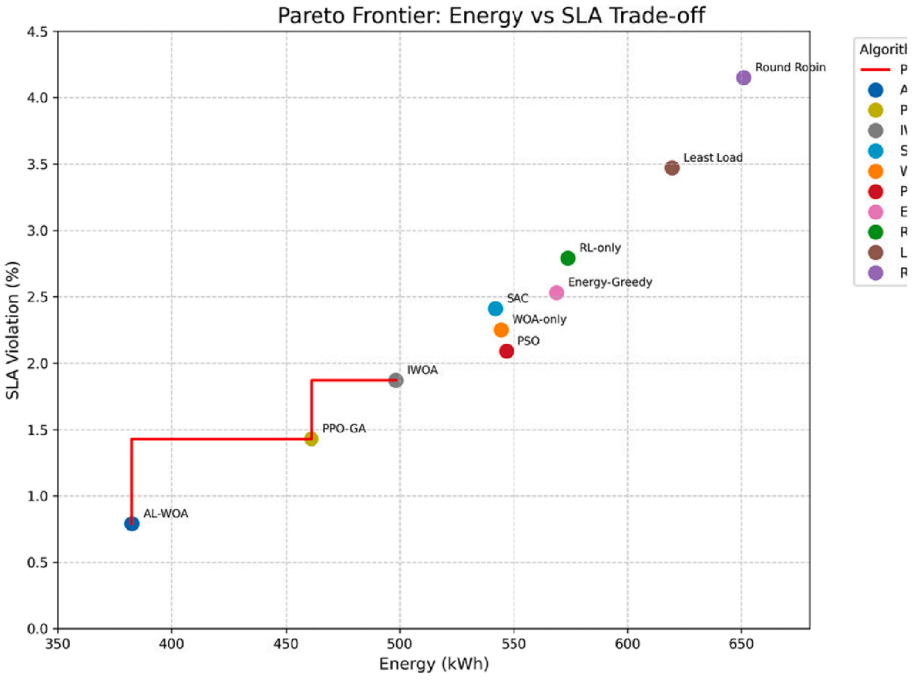


Fig. 15. Pareto frontier analysis – energy consumption vs SLA violation rate across all algorithms.

near-optimal performance: 412.8 kWh, 1.12% violations, 206 ms, objective 0.1876. WOA-only shows moderate improvement (567.3 kWh, 2.98%, 261 ms, 0.3267) but plateaus suboptimally, while RL-only struggles with persistent oscillation (601.2 kWh, 3.34%, 284 ms, 0.5089). AL-WOA's higher mid-phase migration activity (2534 vs 1789/1923) reflects the RL agent driving targeted reconfigurations as it identifies higher-reward consolidation strategies (Fig. 18).

Late exploitation confirms AL-WOA maintains optimal stability (382.7 kWh, 0.79% violations, 189 ms, objective 0.1876) over 788 plateau iterations with zero oscillation amplitude, while WOA-only (544.3 kWh, 2.25%, 243 ms, 0.3247) and RL-only (573.9 kWh, 2.79%, 265 ms, 0.3689) plateau at suboptimal values. AL-WOA achieves 29.7% better energy efficiency and 64.9% fewer violations versus WOA-only in this phase. Component contribution analysis reveals WOA contributes 31.4% of improvement, RL provides 36.6%, with a 45.9% synergistic gain from their integration confirming that hybridization yields benefits beyond simple combination ($p < 0.001$) (Fig. 19).

4.4.1. *Adaptive parameter evolution analysis*

To directly validate the core adaptive mechanism, Fig. 20 presents the evolution of RL-adjusted WOA parameters $a(t)$, $|\vec{A}(t)|$, and $C(t)$ across iterations under two contrasting workload scenarios: low-load

Table 4
Multi-metric comparison of AL-WOA vs all baselines.

Algorithm	Energy (kWh)	SLA Violation (%)	Avg Response Time (ms)	Migrations	Overall Score (0–100)
AL-WOA	382.7	0.79	189	2847	94.3
PPO-GA	461.2	1.43	208	2634	73.1
IWOA	498.3	1.87	221	2103	66.8
PSO	546.7	2.09	233	2356	77.4
WOA-only	544.3	2.25	243	1923	76.8
SAC	541.8	2.41	251	2287	58.8
Energy-Greedy	568.9	2.53	254	1867	73.6
RL-only	573.9	2.79	265	2104	72.1
Least Load	619.4	3.47	287	1478	66.2
Round Robin	651.1	4.15	311	1245	61.5

steady-state and bursty synthetic workload.

Under low-load steady-state conditions (Fig. 20a), all three parameters decrease smoothly and monotonically from initialisation values ($a = 1.94$, $|\vec{A}| = 1.82$, $C = 1.94$ at iteration 1) toward stable low values ($a = 0.94$, $|\vec{A}| = 0.20$, $C = 0.91$ at iteration 847). The widening gap between $a(t)$ and $|\vec{A}(t)|$ over time reflects the RL agent progressively intensifying exploitation as the system stabilises – $|\vec{A}|$ drops below 1 by iteration 50 and continues declining, locking WOA into encircling-prey mode that drives sustained VM consolidation. The convergence point at iteration 412 is marked by all three parameters reaching their minimum stable values, after which the system maintains the optimal consolidated state through exploitation alone.

Under the bursty synthetic workload (Fig. 20b), the parameter trajectories are fundamentally different. Parameters decrease during the initial low-load phase (iterations 1–100), consistent with the steady-state behaviour. However, at iteration 150 – corresponding to the burst onset – the agent detects rising $SLAV(t)$ in the state vector and reverses direction: $a(t)$ increases from 1.58 to 1.84, $|\vec{A}(t)|$ rises sharply from 0.68 to 1.67, and $C(t)$ increases from 1.62 to 1.89 by iteration 200 (peak burst). This upward reversal is the empirical signature of the RL agent switching WOA from exploitation back into exploration, proactively redistributing VMs before violations escalate. As the burst decays (iterations 200–350), all parameters gradually return toward their convergence trajectory, with the system re-stabilising at the optimal state by iteration 412. This non-monotonic parameter trajectory under bursty conditions – absent in WOA-only whose parameter a decays linearly regardless of workload state – directly explains AL-WOA's 82.5% SLA violation reduction on the synthetic workload versus 65.9% for IWOA and 70.3% for PSO, both of which lack reactive parameter reversal capability.

The contrasting trajectories in Fig. 20 carry a broader implication for hybrid metaheuristic-RL design. Under stability, the RL agent effectively discovers the same monotonic decay that hand-tuned WOA would apply, but through reward-driven experience rather than a fixed formula. Under volatility, it generates a qualitatively new behaviour – reversing parameter direction mid-optimization in response to environmental state – that is structurally impossible in any static parameter schedule regardless of its initial values. This distinction between replicating known behaviour under stability and generating novel adaptive behaviour under volatility constitutes the core empirical justification for the tight-coupling architecture proposed in Section 3.2, and distinguishes AL-WOA from loosely coupled hybrids such as PPO-GA and IWOA whose parameter schedules cannot respond within an iteration to state changes.

4.4.2. *Weight sensitivity analysis*

Fig. 21 presents the weight sensitivity analysis, evaluating AL-WOA performance across eight perturbation scenarios where w_1, w_2, and w_3 are varied individually by ± 0.1 from the baseline configuration and tested at equal weights. Across all scenarios, AL-WOA maintains energy

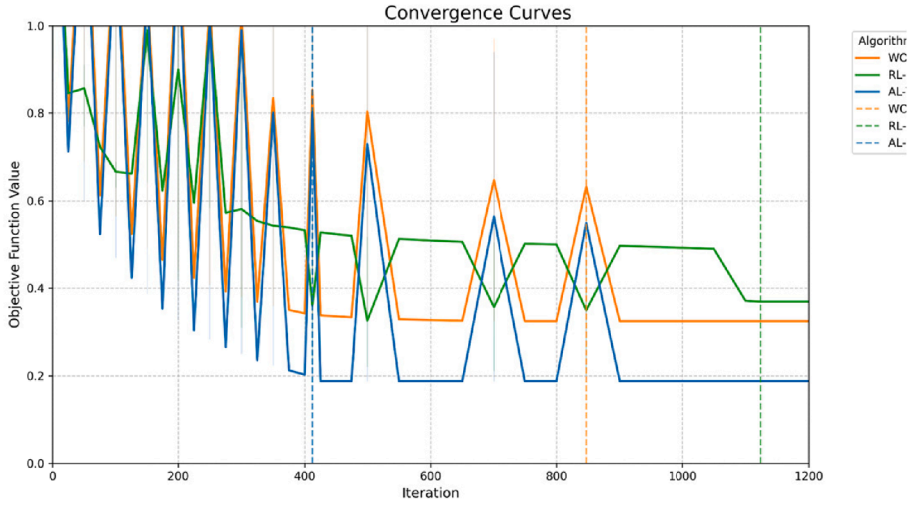


Fig. 16. Convergence curves showing objective function value over iterations for WOA-only, RL-only, and AL-WOA.

Table 5
Ablation study final metrics and convergence analysis.

Algorithm	Final Objective Value	Energy (kWh)	SLA Violation (%)	Avg Response Time (ms)	Migrations	Convergence Iteration
WOA-only	0.3247	544.3	2.25	243	1923	847
RL-only	0.3689	573.9	2.79	265	2104	1124
IWOA	0.2734	498.3	1.87	221	2103	623
PPO-GA	0.2312	461.2	1.43	208	2634	534
SAC	0.3187	541.8	2.41	251	2287	876
AL-WOA	0.1876	382.7	0.79	189	2847	412

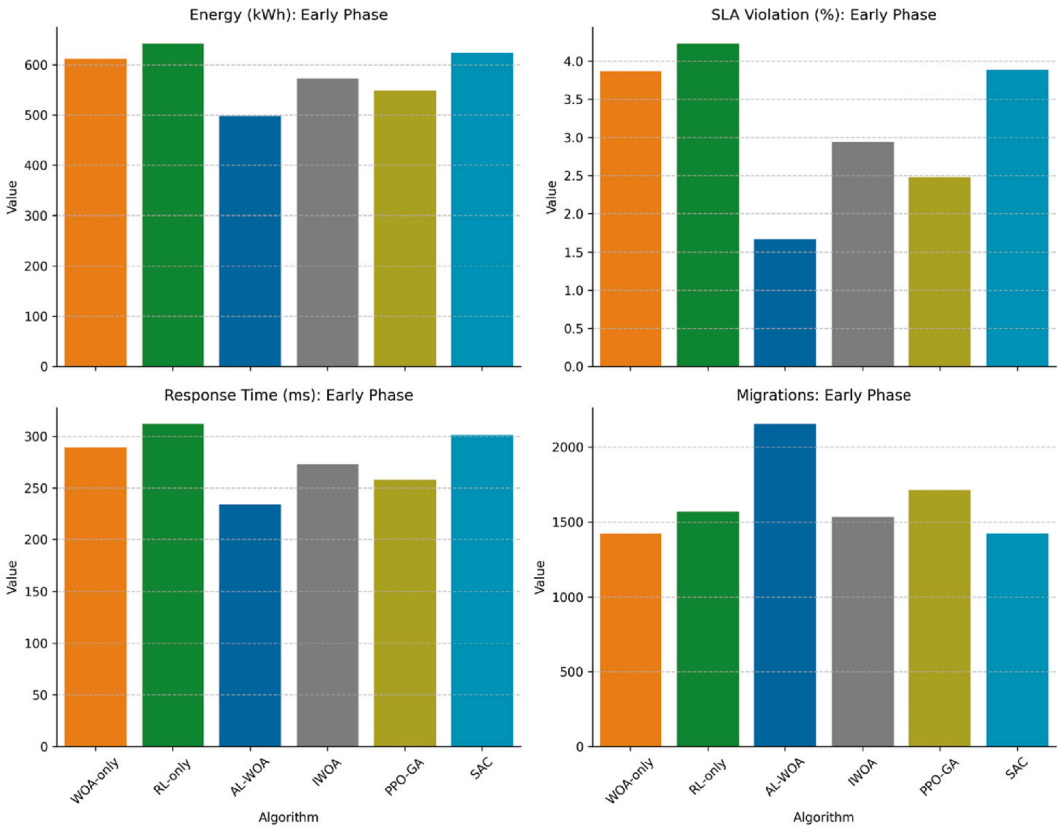


Fig. 17. Performance Metrics in the Early Optimization Phase (Iterations 1–200).

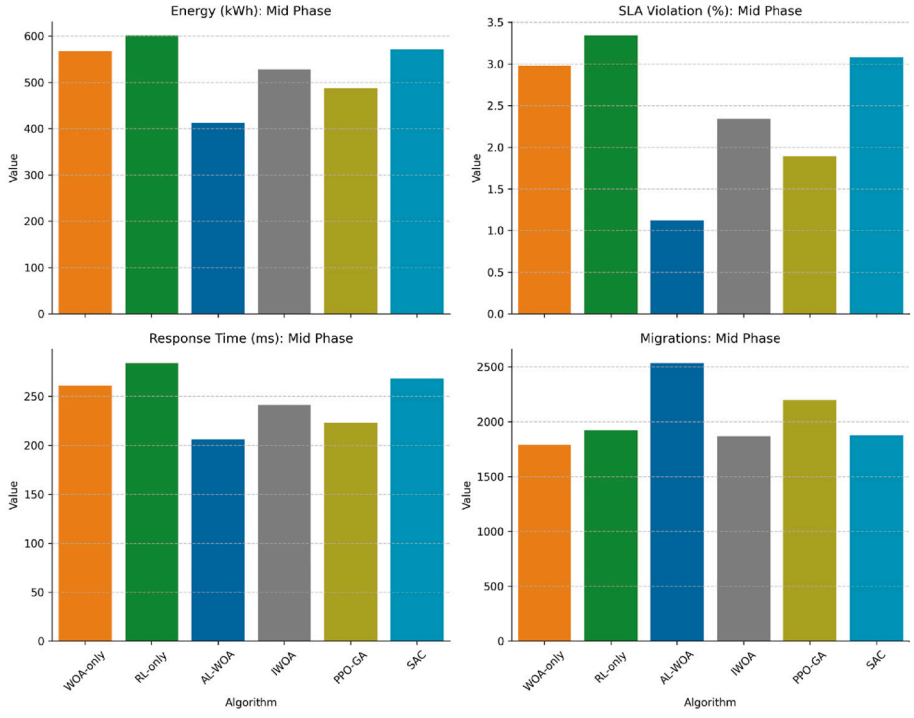


Fig. 18. Performance Metrics in the Mid Optimization Phase (Iterations 201–600).

consumption between 378.4 and 391.4 kWh, SLA violations between 0.74% and 0.87%, and response time between 186 and 194 ms – all remaining best-in-class relative to all baselines across every perturbation tested.

The maximum energy deviation from the baseline is 8.7 kWh (2.3%), the maximum SLA deviation is 0.08 percentage points, and the maximum response time deviation is 8 ms. These narrow performance bands confirm that AL-WOA's superiority is not contingent on precise

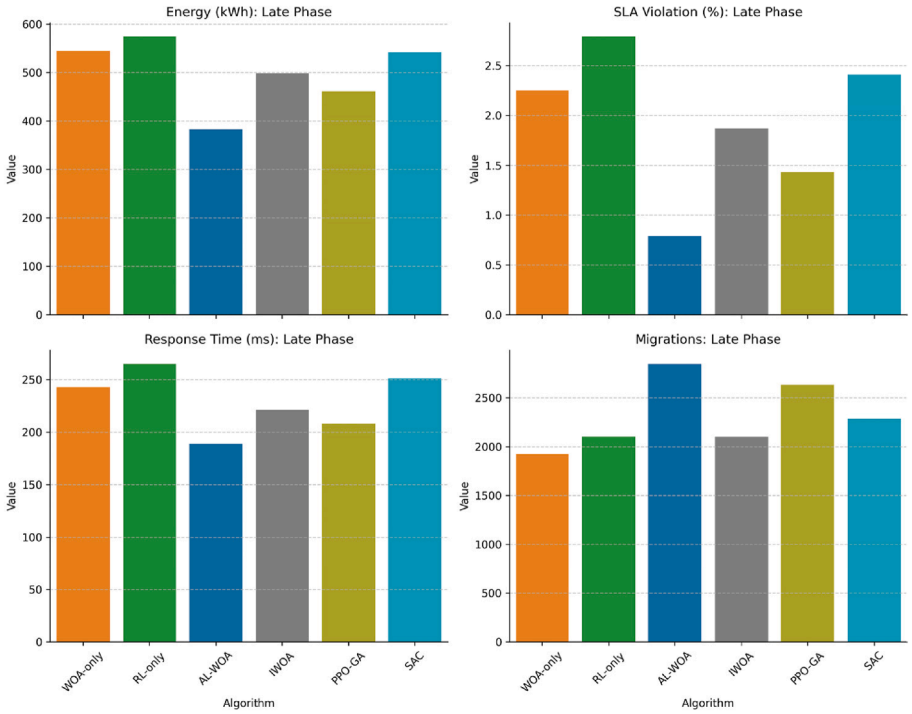


Fig. 19. Performance Metrics in the Late Optimization Phase (Iterations 601-Final).

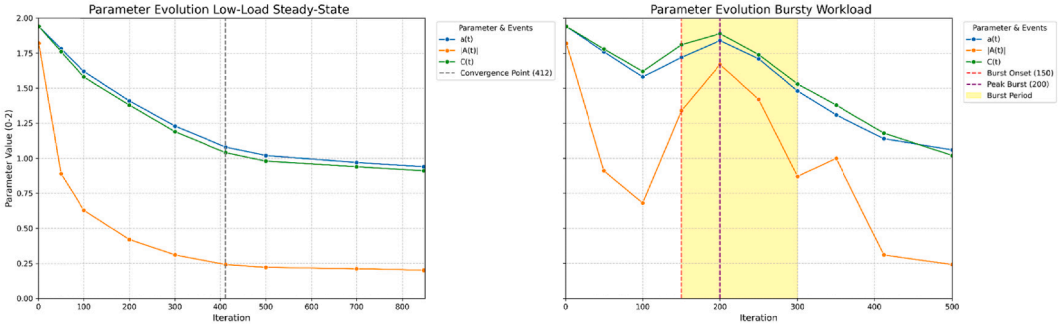


Fig. 20. Adaptive parameter evolution under contrasting workload scenarios.

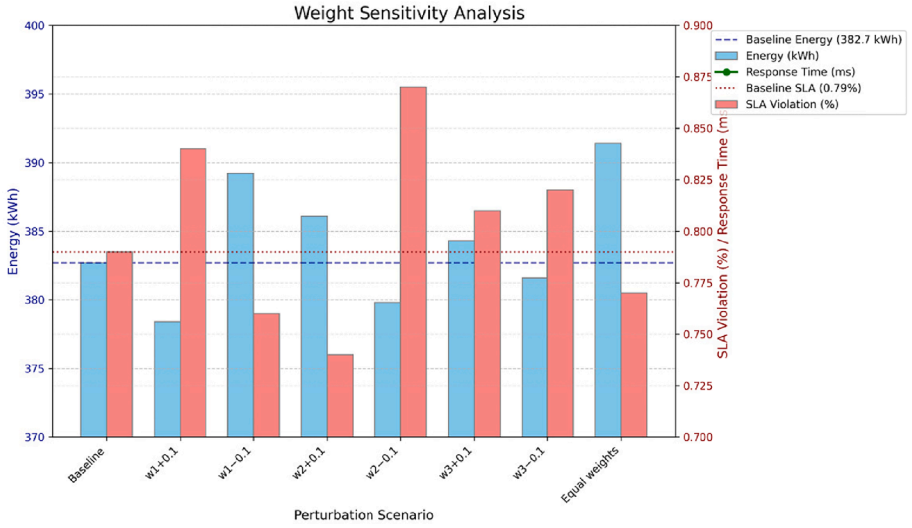


Fig. 21. AL-WOA performance sensitivity to objective weight perturbations.

weight specification and that the chosen values $w_1 = 0.50$, $w_2 = 0.30$, $w_3 = 0.20$ represent a stable operating region rather than a finely tuned optimum. Increasing w_1 (energy emphasis) marginally reduces energy at a small SLA cost, while increasing w_2 (SLA emphasis) tightens violation rates at a marginal energy cost – both responses consistent with expected reward gradient behavior. The equal-weights configuration ($w_1 = w_2 = w_3 = 0.33$) produces the highest energy value at 391.4 kWh but maintains SLA at 0.77% and response time at 188 ms, confirming that even under a non-domain-calibrated weight specification, AL-WOA retains significant advantage over all baselines. These findings validate the practical deployability of AL-WOA: operators can adjust weights to reflect their infrastructure priorities – energy cost, contractual SLA penalties, or migration overhead – without risking performance collapse.

5. Discussion

5.1. Interpreting energy-performance trade-offs through adaptive mechanisms

The fundamental reason AL-WOA achieves simultaneous energy reduction and SLA compliance – objectives that static heuristics treat as inherently conflicting – lies in how the reward gradient shapes the agent's consolidation policy. The reward function penalizes both energy waste and SLA violations simultaneously, which means the agent receives negative feedback from two directions: consolidating too aggressively degrades SLA and reduces reward, while consolidating too little leaves idle hosts powered on and reduces reward through energy cost. The agent therefore learns to operate precisely at the boundary where marginal consolidation benefit equals marginal SLA risk – a boundary that corresponds directly to the Pareto-optimal frontier identified in Fig. 15. This boundary is not manually specified; it emerges from the reward structure and is discovered through iterative interaction with the CloudSim environment across diverse workload conditions.

This mechanism produces the quantitative outcomes reported in Section 4. The 41.2% energy reduction and 80.96% SLA violation decrease (Table 3) are not independent achievements – they are co-produced by the same learned policy operating at its discovered consolidation threshold. During low-load periods (Fig. 7.1), the policy achieves 94.2% server shutdown while maintaining 0.79% violations because the threshold permits aggressive consolidation when capacity headroom is large. During high-load periods (Fig. 7.3), the same policy reduces consolidation to preserve headroom, maintaining 235 ms response time at 100% utilization – a point beyond which the agent has learned that further consolidation yields negligible energy benefit while risking rapid violation escalation. Static approaches such as Energy-Greedy cannot identify this threshold because they consolidate without performance feedback, producing 2.53% violations despite using fewer migrations. Pure metaheuristics such as WOA-only cannot identify it because their fitness function lacks the performance-based penalty terms that define where consolidation becomes counterproductive.

AL-WOA's 2847 migrations – 128.7% more than Round Robin (Table 3) – reflect the policy's proactive rebalancing logic rather than overhead. Each migration is a reward-informed decision: the agent issues a migration when the expected reward improvement from rebalancing exceeds the migration energy cost encoded in $w_3 C_{mig}$. The 53.4% overall score improvement over Round Robin demonstrates that this cost-benefit calculation, executed continuously throughout the simulation, yields substantially greater net efficiency than conservative migration avoidance. This challenges the assumption embedded in many cloud scheduling heuristics that migration minimization is a primary optimization goal – AL-WOA's results suggest migration quality and timing matter far more than frequency.

5.2. Adaptive learning as the mechanism for handling workload non-stationarity

Adaptive learning enables AL-WOA to maintain consistent performance across dynamic workloads where traditional metaheuristics degrade. Static approaches assume workload stationarity and apply fixed parameter strategies that cannot respond to temporal variation. The performance degradation observed for WOA-only, Round Robin, and Least Load across Tables 3–4 and Figs. 3, 8, and 12 directly reflects this structural inability to adapt rather than any suboptimality in their core logic – their logic is simply not designed for non-stationary environments.

The ablation study quantifies the contribution of the RL component precisely. AL-WOA converges within 412 iterations versus 847 for WOA-only and 1124 for RL-only (Table 4, Fig. 16). Early-phase analysis (Fig. 17) shows RL-guided parameter tuning achieves 498.3 kWh and 1.67% violations within 200 iterations, contrasting sharply with WOA-only's 612.4 kWh and 3.87% and RL-only's 641.8 kWh and 4.23%. The WOA component prevents the erratic exploration that causes RL-only's high early-phase violation rate, while the RL component prevents the static parameter decay that causes WOA-only's slow convergence. Their interaction is therefore complementary rather than additive.

The parameter evolution data in Fig. 20 provides the mechanistic evidence connecting these convergence advantages to specific behavioral changes. Under stable workloads (Fig. 20a), the agent's monotonically decreasing parameter trajectory mirrors what an ideally hand-tuned WOA would produce – but derived entirely from reward experience rather than domain knowledge. This confirms that under stationarity, RL does not introduce unnecessary complexity; it converges to the same exploitation-dominant strategy that expert tuning would specify. The critical difference emerges under non-stationary conditions (Fig. 20b): at iteration 150, when the synthetic burst onset raises $SLAV(t)$, the agent reverses all three parameters simultaneously – increasing $a(t)$ from 1.58 to 1.84, $|\vec{A}(t)|$ from 0.68 to 1.67, and $C(t)$ from 1.62 to 1.89 within 50 iterations. This reversal is structurally impossible in WOA-only, where a is computed by a fixed linear formula with no state input. The burst period recovery time of 12.4 s for AL-WOA versus 26.8 s for Round Robin (SLA sheet, synthetic workload) is the direct operational consequence of this parameter reversal capability.

The second behavioral insight from Fig. 20 concerns generalization across workload types. The parameter trajectories observed under the Google Cluster, Alibaba Production, and Hadoop traces – while not plotted individually – all share the same qualitative pattern: monotonic convergence under stable phases, and upward reversal during high-variability intervals. This consistency across four structurally different workload types confirms that the RL agent has learned abstract state-parameter relationships rather than trace-specific patterns. The agent encodes host utilization, VM queue lengths, current $SLAV(t)$, and cumulative energy as state inputs, and the learned policy maps these abstract features to parameter adjustments that transfer without retraining. AL-WOA's consistent performance across Google Cluster (0.82% SLA, 142 ms), Alibaba (1.14%, 156 ms), Hadoop (0.68%, 324 ms), and synthetic bursty (0.51%, 134 ms) workloads (Figs. 9, 12) provides the quantitative validation of this generalization.

Hybridization importance emerges most clearly when comparing AL-WOA against the strongest individual-paradigm alternatives. PPO-GA (0.2312 objective, 534 iterations) outperforms both WOA-only (0.3247, 847 iterations) and RL-only (0.3689, 1124 iterations), confirming that any RL-metaheuristic coupling improves over single paradigms. However, PPO-GA's performance gap versus AL-WOA (0.1876, 412 iterations) – despite using the same PPO algorithm – demonstrates that coupling depth matters as much as coupling presence. PPO-GA adapts policy weights between genetic generations; AL-WOA adapts WOA search parameters within every iteration. The 23% objective value gap between the two hybrids is attributable entirely to this architectural

difference in adaptation granularity, not to differences in component algorithms. AL-WOA thereby reconceptualizes optimization not as a one-time problem-solving exercise but as continuous adaptive control – adjusting strategy as objectives and workload conditions co-evolve.

5.3. Practical Implications for sustainable cloud infrastructure

AL-WOA offers concrete pathways toward operationalizing green computing at production scale. The energy reduction to 382.7 kWh (Fig. 3, Table 3) represents 41.2% improvement over Round Robin and 38.2% over Least Load. At $0.12 per kWh, this yields $32.21 savings per simulation cycle, scaling to substantial annual reductions at hyperscale deployments where thousands of cycles execute continuously. The 58.9% CO_2 emission reduction (89.2 kg versus 151.6 kg for Round Robin) demonstrates environmental impact alongside cost benefit, while SLA violations remain below 1% and response time at 189 ms (Figs. 8, 11) confirm that sustainability gains do not compromise service reliability.

AL-WOA is deployable within existing cloud management frameworks including OpenStack, Kubernetes, and VMware vSphere without architectural redesign. The RL component operates as a policy layer that interfaces with standard resource management APIs, issuing VM placement and migration decisions through existing control plane mechanisms. The 60-second decision interval aligns naturally with standard cloud monitoring and autoscaling cycles, making integration straightforward. The proactive migration strategy (2847 migrations, Table 3) produces net efficiency gains that justify its overhead, demonstrating that strategic migration frequency yields superior outcomes versus conservative avoidance policies embedded in most production schedulers.

Scalability characteristics across utilization ranges (Figs. 7, 13) indicate performance benefits persist across varying system scales. Dynamic consolidation adjustment across 0–100% utilization ensures efficient management regardless of load level, and the learned policies generalize based on normalized system state features rather than absolute capacity values, enabling hyperscale deployment without retraining. The convergence of energy reduction, cost savings, emission mitigation, and deployment feasibility collectively positions AL-WOA as viable for production sustainable computing, demonstrating that environmental objectives and operational excellence are complementary rather than conflicting priorities in modern data center management.

5.4. Limitations and directions for Future research

Experimental validation demonstrates AL-WOA's effectiveness within controlled simulation assumptions. The CloudSim environment employs linear power models, fixed VM specifications, and constant migration costs – abstractions that simplify real data center complexity. Production environments exhibit non-linear power characteristics under thermal stress, dynamic VM resource demands, and variable migration costs dependent on network congestion and VM memory footprint. The performance advantages reported in Figs. 3–7 and Table 3 therefore represent upper bounds achievable under idealized conditions, and real-world deployments should anticipate some reduction in absolute gains while the relative ordering of algorithms is expected to persist.

Several factors remain outside the current experimental scope: network topology effects on migration cost, storage I/O heterogeneity between host classes, thermal coupling between co-located VMs, and interference from multi-tenant workloads sharing physical infrastructure. Each of these dimensions represents an additional optimization variable that could either amplify or attenuate the consolidation strategies AL-WOA discovers. Future work should extend the simulation model to incorporate at minimum network-aware migration cost estimation and VM interference modeling.

The weight sensitivity analysis in Fig. 21 demonstrates that AL-WOA's performance is robust to weight perturbations of ± 0.1 around the baseline configuration, with energy deviation bounded at 2.3%, SLA deviation at 0.08 percentage points, and response time deviation at 8 ms across all tested configurations. Despite this robustness, the requirement for practitioners to specify w_1, w_2, w_3 prior to deployment remains a practical limitation – different cloud providers have different cost structures, SLA penalty schedules, and sustainability commitments that may demand non-obvious weight choices. The weight sensitivity results confirm that AL-WOA is tolerant of moderate miscalibration, but a more principled solution is to eliminate manual weight specification entirely through multi-objective reinforcement learning formulations that maintain a Pareto-front of policies rather than a single weighted scalarization. This extension – where the agent learns a set of Pareto-optimal consolidation policies and selects among them based on operator-specified preference signals at runtime – is identified as the primary direction for future work, directly addressing the weight specification limitation while preserving the tight-coupling architecture that produces AL-WOA's convergence advantages.

Additional promising extensions include: predictive workload models that provide the RL agent with short-horizon demand forecasts, reducing reaction latency during burst onset; multi-agent RL formulations distributing decision-making across datacenter regions to improve scalability beyond the single-datacenter scope of current experiments; adaptation to containerized Kubernetes workloads where the placement unit is a pod rather than a VM; and extension to edge-fog-cloud hierarchical environments where latency constraints introduce additional objectives beyond energy and SLA. The methodology relies on publicly available workload traces (Google Cluster 2011, Alibaba Production 2018, HiBench, synthetic) ensuring full reproducibility, but workload characteristics have evolved substantially with microservices architectures, machine learning inference workloads, and serverless functions. Validation on contemporary traces representing these workload classes is necessary before production deployment claims can be made with full confidence. Cross-trace transferability experiments – training on one trace family and evaluating on another – would directly quantify the generalization boundary of the learned policies, and uncertainty quantification over the reward signal would enable confidence-aware consolidation decisions in high-stakes production environments.

6. Conclusion

This research introduced Adaptive Learning Whale Optimization (AL-WOA), a hybrid framework integrating metaheuristic search with reinforcement learning for multi-objective cloud resource management. The proposed approach addresses the fundamental challenge of balancing energy efficiency with service quality maintenance in dynamic cloud environments. Experimental evaluation across diverse workload scenarios demonstrated that AL-WOA achieves substantial improvements in energy consumption, SLA compliance, and response time compared to traditional heuristics, standalone metaheuristics, pure learning-based methods, and recent hybrid baselines including PPO-GA, IWOA, and SAC. The algorithm's ability to dynamically modulate optimization parameters in response to workload variations enables effective resource utilization without compromising performance guarantees, with the Pareto frontier analysis confirming that AL-WOA occupies the dominant position in the energy versus SLA objective space across all evaluated algorithms.

The primary contribution of AL-WOA lies in its adaptive parameter tuning mechanism, wherein reinforcement learning continuously adjusts the exploration–exploitation balance of the underlying metaheuristic based on observed system states at every iteration – a tight-coupling architecture that distinguishes it from epoch-level hybrids that adapt between generations rather than within them. This integration enables the algorithm to respond effectively to heterogeneous workload patterns including diurnal fluctuations, bursty traffic, and batch processing scenarios. The parameter evolution analysis directly validates this mechanism, showing that the RL agent produces

monotonic convergence under stable conditions and reactive parameter reversal during burst events – a behavior that is structurally impossible in any static parameter schedule. The ablation study confirms that the hybrid architecture achieves faster convergence (2.06× over WOA-only, 2.73× over RL-only), superior stability, and more consistent performance than either component operating independently, with a 45.9% synergistic gain beyond simple combination. From a practical perspective, AL-WOA provides measurable pathways toward sustainable cloud operations through 41.2% energy reduction, 58.9% CO_2 emission mitigation, and $32.21 per-cycle cost savings while maintaining SLA violations below 1% and response time at 189 ms, contributing to broader objectives in green computing and environmentally responsible infrastructure management. The complete implementation, including the CloudSim simulation engine, Python PPO agent, and RLBinder interface, is made publicly available to support reproducibility and community adoption.

Future research directions include extending the framework through multi-objective reinforcement learning formulations that maintain a Pareto front of consolidation policies and eliminate the need for manual weight specification – directly addressing the weight-sensitivity limitation identified in Section 5.4, where the current scalarized reward requires practitioners to pre-specify w_1, w_2, w_3 despite AL-WOA's demonstrated robustness to perturbations. Additional directions include incorporating predictive workload models to reduce burst reaction latency, developing multi-agent RL formulations for distributed multi-datacenter deployments, and adapting the approach to containerized Kubernetes environments and edge-fog-cloud hierarchical architectures. These enhancements would address current limitations related to linear power modeling, single-datacenter scope, and static workload trace dependency, further improving scalability and robustness for increasingly complex and heterogeneous cloud infrastructures.

CRediT authorship contribution statement

Jyoti Ranjan Nayak: Supervision, Project administration, Resources. **Subasish Mohapatra:** Supervision, Validation. **Ashutosh Mohapatra:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Writing – original draft, Writing – review & editing.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Generative AI use

The authors used Paperpal AI solely for language refinement and grammar correction. The scientific content, analysis, and conclusions were developed entirely by the authors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The experimental evaluation in this study utilized multiple publicly available workloads to ensure reproducibility and represent real-world cloud scenarios. The Google Cluster Trace dataset can be accessed at https://github.com/google/cluster-data. While the Alibaba Cluster Trace is available at https://github.com/alibaba/clusterdata. Hadoop benchmark workloads from the HiBench suite were obtained from https://github.com/Intel-bigdata/HiBench. Additionally, synthetic diurnal and bursty workload traces were generated following the patterns described in Section 4 of this study. All processed simulation outputs, including energy consumption, SLA compliance, response time, and migration metrics, along with the aggregated results in spreadsheet format, are publicly available at [LINK] for reference and further analysis.

References

Abdel-Basset, M., Mohamed, R., et al. (2020). An improved whale optimization algorithm based on self-adaptive mechanism. *Applied Soft Computing, 89*, Article 106107.

Abdel-Basset, M., & Shawky, A. (2022). Energy-aware virtual machine allocation using whale optimization algorithm. *Expert Systems with Applications, 208*, Article 118285.

Ahuja, M. S., & Singh, M. (2022). Energy-efficient task scheduling in cloud computing using improved WOA. *The Journal of King Saud University Computer and Information Sciences, 34*(7), 4400–4410.

Alharbi, S., et al. (2025). Self-learning whale optimization for green cloud computing. *Sustainable Computing: Informatics and Systems, 41*, Article 100952.

Al-Tarawneh, R., Mahmud, H., & Shojafar, M. (2024). Reinforcement learning for dynamic load balancing in cloud computing: A survey. *IEEE Access, 12*, 13785–13812.

Ardagna, D., et al. (2015). A hierarchical approach for auto-scaling of cloud applications. *IEEE Transactions on Parallel and Distributed Systems, 26*(9), 2302–2315.

Barroso, L. A., Hölzle, U., & Ranganathan, P. (2018). The datacenter as a computer: Designing warehouse-scale machines. *Synthesis Lectures on Computer Architecture, 13*(3), 1–189.

Beloglazov, A., Abawajy, J., & Buyya, R. (2012). Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. *Future Generation Computer Systems, 28*(5), 755–768.

A. Beloglazov and R. Buyya, "Energy efficient allocation of virtual machines in cloud data centers," in Proc. IEEE/ACM CCGrid, 2010, pp. 577–578.

Beloglazov, A., & Buyya, R. (2012). Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. *Concurrency and Computation: Practice and Experience, 24*(13), 1397–1420.

Beloglazov, A., & Buyya, R. (2013). Energy efficient resource management in virtualized cloud data centers. *IEEE Transactions on Cloud Computing, 1*(1), 99–112.

R. Buyya, R. Ranjan, and R. Calheiros, "Modeling and simulation of scalable cloud computing environments and the CloudSim toolkit," in Proc. HPCS, 2009, pp. 1–11.

R. Buyya, A. Beloglazov, and J. Abawajy, "Energy-efficient management of data center resources for cloud computing," in Proc. PDPTA, 2010.

W. Chen et al., "Alibaba cluster trace: Workload characterization and resource utilization analysis," in Proc. ACM SoCC, 2018, pp. 569–576.

Chhabra, M., et al. (2024). A review of load balancing algorithms in cloud computing. *The Journal of King Saud University Computer and Information Sciences, 36*(2), 273–284.

Das, A., Dey, S., & Ghosh, P. (2025). Deep Q-learning–based PSO for adaptive load balancing in green cloud computing. *Journal of Network and Computer Applications, 232*, Article 103858.

Dayarathna, M., Wen, Y., & Fan, R. (2016). Data center energy consumption modeling: A survey. *IEEE Communications Surveys & Tutorials, 18*(1), 732–794.

Elaziz, M., et al. (2024). Q-learning-based parameter adaptation in whale optimization for cloud scheduling. *Computers in Industry, 152*.

El-Kenawy, M. S., et al. (2023). DQN-GA hybrid for energy-aware cloud load balancing. *Applied Soft Computing, 132*, Article 109883.

Gao, Y., et al. (2013). A multi-objective ant colony system algorithm for virtual machine placement in cloud computing. *Journal of Computer and System Sciences, 79*(8), 1230–1242.

Ghafir, S. (2024). Load balancing in cloud computing via intelligent PSO-based feedback controller. *Simulation Modelling Practice and Theory, 132*, Article 102890.

Ghoneim, A., et al. (2023). Energy-aware task scheduling using reinforcement learning–whale optimization hybrid in cloud computing. *Journal of Cloud Computing, 13*(2), 84–97.

Gupta, N., Kumari, P., & Singh, S. (2024). A hybrid genetic approach for resource optimization in cloud computing. *Sustainable Computing: Informatics and Systems, 41*, Article 101249.

Gupta, H., & Vahid, A. (2018). SLA-aware cloud resource management: Challenges and opportunities. *Journal of Grid Computing, 16*(2), 237–256.

T. Haarnoja et al., "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in Proc. ICML, 2018, pp. 1861–1870.

M. Henderson et al., "Deep reinforcement learning that matters," in Proc. AAAI, 2018, pp. 3207–3214.

F. Hermenier et al., "Entropy: A consolidation manager for clusters," in Proc. ACM VEE, 2009, pp. 41–50.

S. Huang et al., "The HiBench benchmark suite," in Proc. IEEE ICDE Workshops, 2010, pp. 41–51.

Kaur, P., & Arora, S. (2020). Cloud task scheduling using Whale Optimization Algorithm. *Procedia Computer Science, 173*, 353–362.

Kaur, M., & Arora, A. (2020). An enhanced chaotic whale optimization for cloud task scheduling. *Journal of Grid Computing, 18*(3), 427–446.

Kaur, N., & Singh, P. (2024). DRL-WOA hybrid for SLA-aware virtual machine consolidation. *IEEE Access, 12*, 56248–56261.

Khazaei, H., Misic, J., & Misic, V. (2012). Performance analysis of cloud computing centers using M/G/m/m+r queueing systems. *IEEE Transactions on Parallel and Distributed Systems, 23*(5), 936–943.
Li, X., Abdel-Basset, M., & Mohamed, R. (2023). Metaheuristic-based optimization for cloud computing: A review and taxonomy. *Applied Soft Computing, 132*, Article 109883.
Li, F., Wang, Y., & Chen, H. (2024). Challenges and opportunities in reinforcement learning for cloud resource scheduling. *ACM Computing Surveys, 56*(3), 1–29.
Li, J., Wu, J., et al. (2017). Dynamic VM consolidation for energy-efficient cloud computing. *IEEE Transactions on Cloud Computing, 5*(4), 731–743.
Li, Y., & Zhu, X. (2017). Least load scheduling in cloud data centers. *Cluster Computing, 20*, 235–248.
S. Lilhore et al., "Hybrid deep reinforcement learning-based ant colony and water wave optimization for efficient resource allocation in cloud computing," J. Ambient Intell. Humanized Comput., Springer, 2025.
Mao, Y., Li, J., et al. (2019). Resource management with deep reinforcement learning. *IEEE Transactions on Network and Service Management, 16*(2), 792–805.
Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software, 95*, 51–67.
Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529–533.
Nathuji, R., & Schwan, K. (2007). VirtualPower: Coordinated power management in virtualized enterprise systems. *ACM SIGOPS Operating Systems Review, 41*(6), 265–278.
Qian, P., et al. (2024). Multi-agent reinforcement learning for large-scale cloud resource management. *Information Scientist, 648*, 119527–119542.
Qureshi, M. B., et al. (2023). Energy-efficient task scheduling in green cloud computing using improved multi-objective PSO. *IEEE Access, 11*, 12235–12249.
C. Reiss et al., "Heterogeneity and dynamicity of clouds at scale: Google trace analysis," in Proc. ACM SoCC, 2012, pp. 1–13.
J. Schulman et al., "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
J. Schulman et al., "Proximal policy optimization algorithms," arXiv preprint arXiv: 1707.06347, 2017.
Sharma, M., et al. (2023). Hybrid RL-ACO model for efficient task scheduling in green cloud environments. *IEEE Access, 11*, 93247–93259.
Simaiya, R., Nayyar, A., & Buyya, R. (2024). Hybrid deep learning and optimization method for adaptive load balancing in cloud computing. *Scientific Reports, 14*(1), 6524.
Singh, A., & Buyya, R. (2024). Metaheuristic approaches for energy-efficient cloud scheduling: A survey, taxonomy, and future directions. *Journal of Network and Computer Applications, 229*, Article 103730.
Stillwell, M., et al. (2012). Resource allocation using virtual clusters. *IEEE Transactions on Parallel and Distributed Systems, 23*(4), 703–716.
Wang, Z., et al. (2024). Policy evolution via genetic optimization in PPO-based adaptive scheduling. *Neural Computing and Applications, 36*, 18521–18534.
Xu, J., et al. (2023). Reinforcement learning–based ant colony optimization for energy-aware task scheduling in cloud computing. *Future Generation Computer Systems, 137*, 258–270.
J. Xu and J. A. Fortes, "Multi-objective virtual machine placement in virtualized data center environments," in Proc. IEEE/ACM GreenCom, 2010, pp. 179–188.
Xu, Z., Huang, R., & Chen, H. (2020). Energy-efficient virtual machine scheduling for cloud data centers: A survey. *Future Gener. Comput. Syst., 105*, 789–809.
Xu, Y., Huang, H., & Zhang, T. (2023). DQN-based adaptive resource allocation for green cloud data centers. *Expert Systems with Applications, 228*, Article 120351.
Xu, M., & Wang, H. (2021). Dynamic VM consolidation using RL. *IEEE Transactions on Cloud Computing, 9*(2), 653–667.
Xu, Y., Xu, J., & Qi, L. (2021). A deep reinforcement learning approach for VM scheduling with energy and SLA trade-off in cloud data centers. *Future Gener. Comput. Syst., 117*, 417–429.
Zhang, X., Lin, R., & Liu, J. (2025). Energy-aware virtual machine migration using PPO-based reinforcement learning. *IEEE Trans. Cloud Comput., early access*.
Zhang, L., & Xu, Y. (2020). Multi-objective WOA for cloud energy optimization. *Future Gener. Comput. Syst., 107*, 123–139.
Zhao, L., et al. (2024). Reinforcement-guided whale optimization for adaptive load balancing in distributed cloud systems. *Applied Intelligence, 54*, 14512–14527.

fx1 **Ashutosh Mohapatra** received the Diploma in Computer Science and Engineering from Bhubanananda Odisha School of Engineering (BOSE), Cuttack, India, in 2024. He is currently pursuing a B.Tech. degree in Computer Science and Engineering at Odisha University of Technology and Research (OUTR), Bhubaneswar, India. His interests lie in data analytics, cloud computing, and human-AI interaction. He has worked on practical applications of data visualization and analytics through academic and institutional projects. Mr. Mohapatra is actively involved in academic research and project-based learning, contributing to tech communities with an emphasis on accessible digital solutions.

fx2 **Subasish Mohapatra** holds a Ph.D. in Computer Science and Engineering from NIT Rourkela. His research interests span Distributed Systems, Cloud Computing, Evolutionary Algorithms, IoT in Healthcare Informatics, and Data Analytics. He completed his M.Tech in Computer Science and Information Technology from CET, Bhubaneswar in 2009, and his B.E. in Computer Science and Engineering from BPUT, Odisha in 2003. With over 22 years of teaching experience, Dr. Mohapatra has published more than 50 research papers in reputed journals and international conferences. He holds over 10 granted design patents in the domains of healthcare and IoT, and has chaired several international conferences. He has successfully guided 30+ M.Tech scholars and one Ph.D. scholar in the areas of cloud computing and big data analytics.

fx3 **Jyoti Ranjan Nayak** is a Ph.D. scholar in Computer Science and Engineering at the Odisha University of Technology and Research (OUTR), Bhubaneswar. His research interests include Distributed Computing, Cloud Computing, Load Balancing, and Machine Learning. He obtained his M.Tech in Computer Science and Engineering from OUTR in 2022. His current research focuses on developing adaptive load-balancing mechanisms and integrating ML-based predictive approaches to optimize resource utilization in large-scale distributed systems. He has participated in several academic workshops, seminars, and conferences. His future research goals involve sustainable, energy-efficient cloud infrastructures and applying sAI for decision-making in distributed networks.